Efficient Query Routing Harnessing LLMs as Retrievers

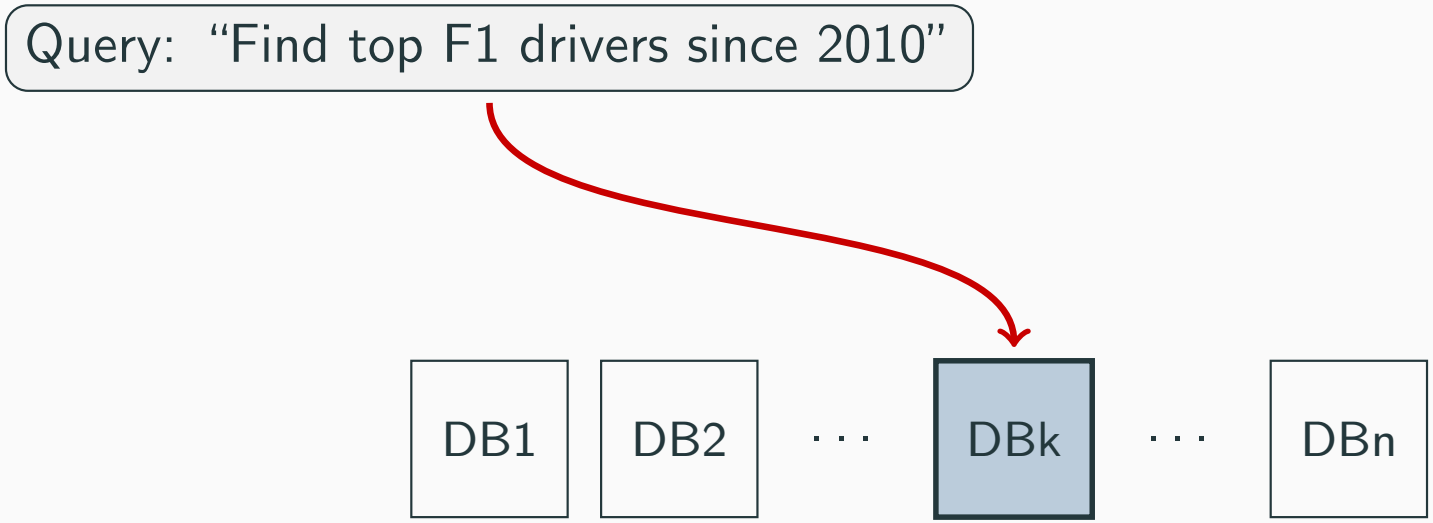
Gaurav Kumar (24M0786)

MTP Stage 1 Presentation

Department of Computer Science and Engineering
Indian Institute of Technology, Bombay
Under the guidance of **Prof. Sunita Sarawagi**

Introduction

The Problem: A Needle in a Haystack



The Challenge

Given a natural language query and hundreds of potential databases, how do we identify the single correct database schema with high accuracy and efficiency?

Our Approach: Harnessing Semantic Understanding

The Problem's Core Difficulty

Simple keyword matching fails because it misses **semantic intent**. A query for "highest earning movies" might need a database with columns like 'revenue' or 'box_office', not just the word "earning".

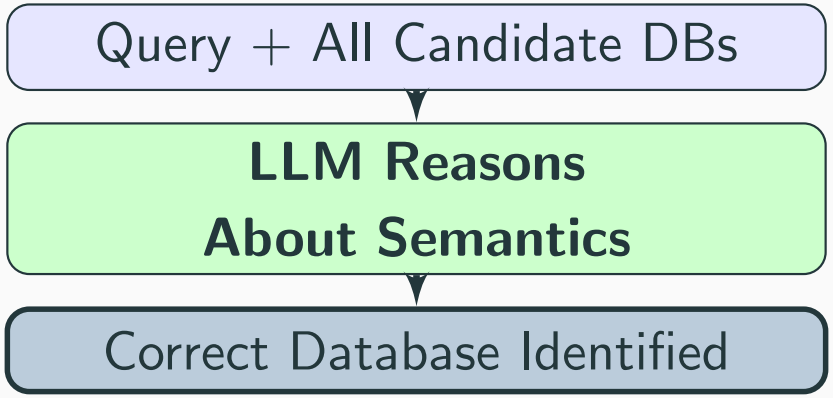
Our Approach: Harnessing Semantic Understanding

The Problem's Core Difficulty

Simple keyword matching fails because it misses **semantic intent**. A query for "highest earning movies" might need a database with columns like 'revenue' or 'box_office', not just the word "earning".

The Solution: Using an LLM as a Semantic Router

Large Language Models don't just match words; they understand the *meaning* behind them. We can leverage this to build an intelligent router.



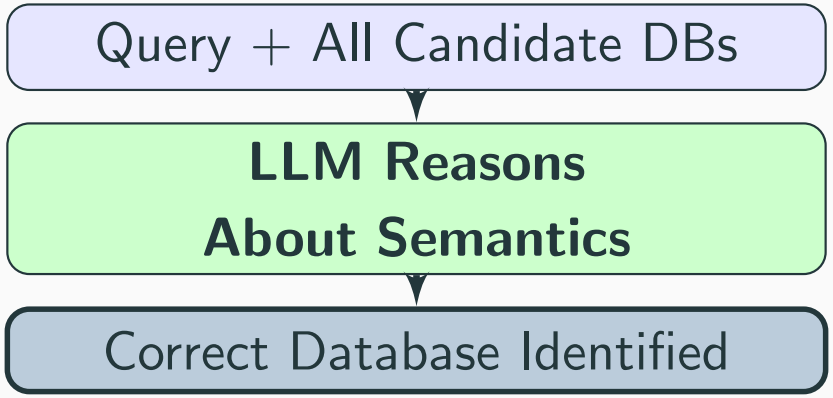
Our Approach: Harnessing Semantic Understanding

The Problem's Core Difficulty

Simple keyword matching fails because it misses **semantic intent**. A query for "highest earning movies" might need a database with columns like 'revenue' or 'box_office', not just the word "earning".

The Solution: Using an LLM as a Semantic Router

Large Language Models don't just match words; they understand the *meaning* behind them. We can leverage this to build an intelligent router.



This Motivates Our Core Investigation

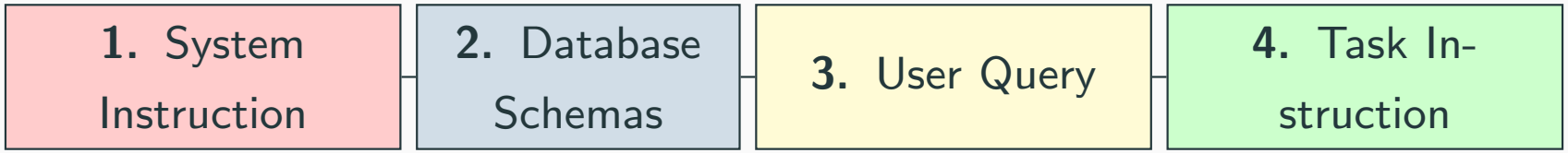
To use this powerful tool effectively, we *must* first look inside the "black box" and ask: How does it really work, and what are its hidden biases?

Part 1: Inside the LLM's Mind

Our Experimental Setup: A Controlled Environment

High-Level Prompt Structure

Every prompt we use follows the same high-level, four-part structure to ensure consistency:



The Core Experiment: The "Positional Sweep"

Our main experiment focuses on the **Database Schemas** block.

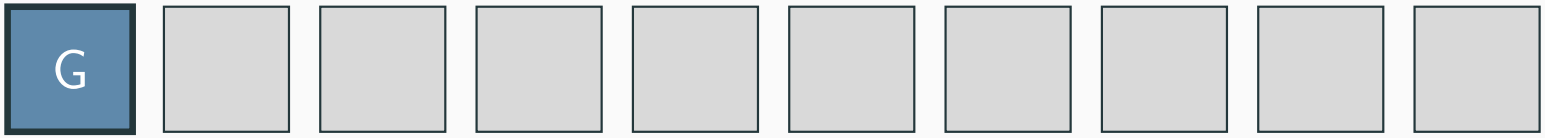
- This block always contains **30 schemas**: 1 correct (gold) and 29 incorrect (distractor).
- We systematically move the position of the single gold schema from 1 to 30 across different runs to measure how its location impacts the model's attention.

Experiment : The “Needle in a Haystack” Test

The Controlled Setup

For every experiment, we create a prompt with a fixed structure to isolate the effect of position:

- **Total Schemas:** 30
- **The “Needle”:** 1 correct **(gold)** database schema.
- **The “Haystack”:** 29 irrelevant **(distractor)** database schemas.

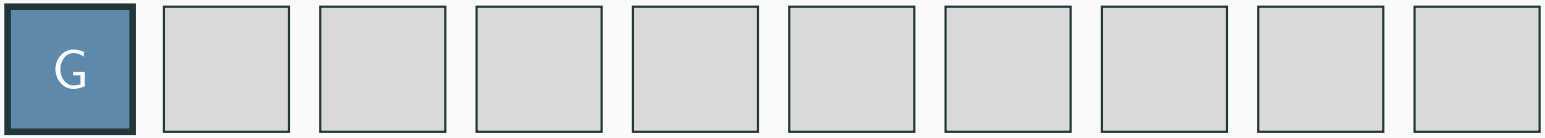


Experiment : The “Needle in a Haystack” Test

The Controlled Setup

For every experiment, we create a prompt with a fixed structure to isolate the effect of position:

- **Total Schemas:** 30
- **The “Needle”:** 1 correct **(gold)** database schema.
- **The “Haystack”:** 29 irrelevant **(distractor)** database schemas.

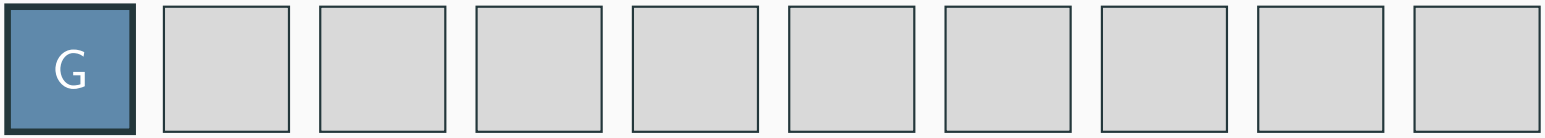


Experiment : The “Needle in a Haystack” Test

The Controlled Setup

For every experiment, we create a prompt with a fixed structure to isolate the effect of position:

- **Total Schemas:** 30
- **The “Needle”:** 1 correct **(gold)** database schema.
- **The “Haystack”:** 29 irrelevant **(distractor)** database schemas.

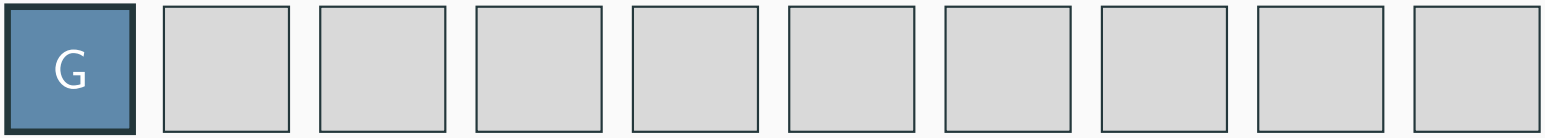


Experiment : The "Needle in a Haystack" Test

The Controlled Setup

For every experiment, we create a prompt with a fixed structure to isolate the effect of position:

- **Total Schemas:** 30
- **The "Needle":** 1 correct **(gold)** database schema.
- **The "Haystack":** 29 irrelevant **(distractor)** database schemas.

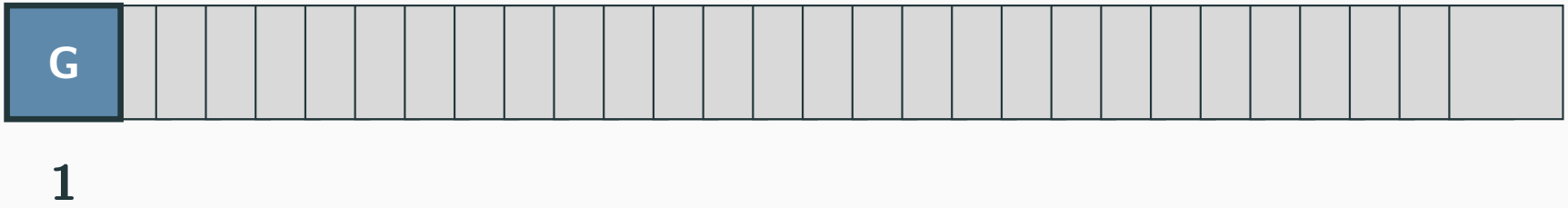


The Core Question

If we move the **gold** database, how does that affect the model's attention? Does it pay more attention to certain positions?

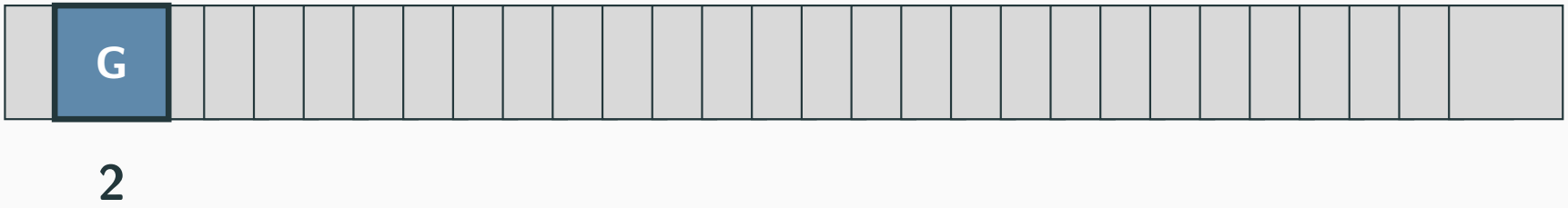
The Process: A Systematic Positional Sweep

Run 1: Gold DB at Position 1



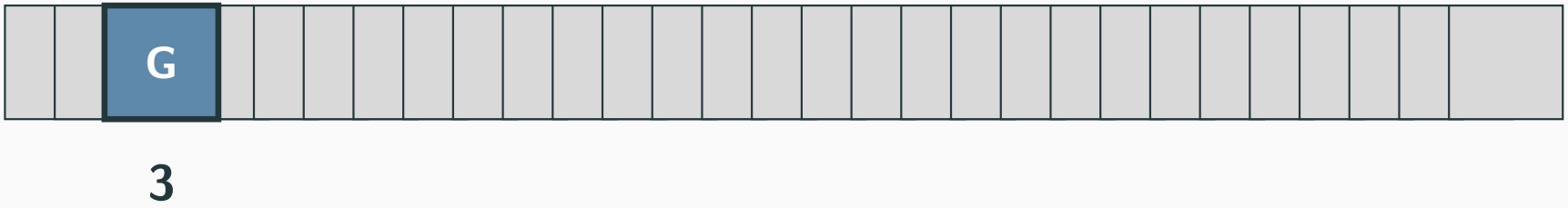
The Process: A Systematic Positional Sweep

Run 2: Gold DB at Position 2



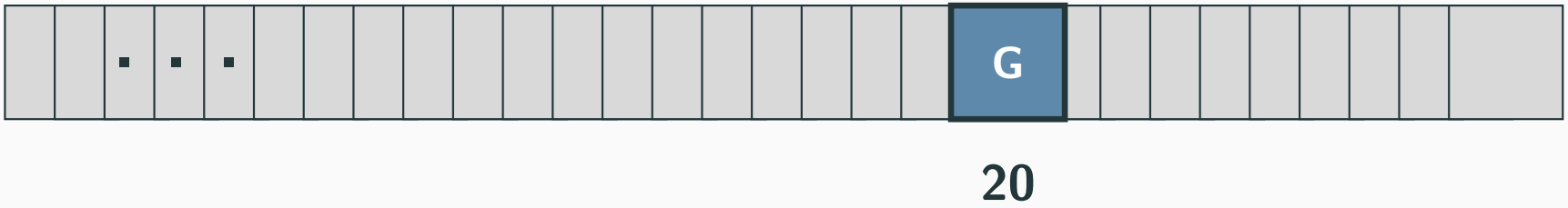
The Process: A Systematic Positional Sweep

Run 3: Gold DB at Position 3



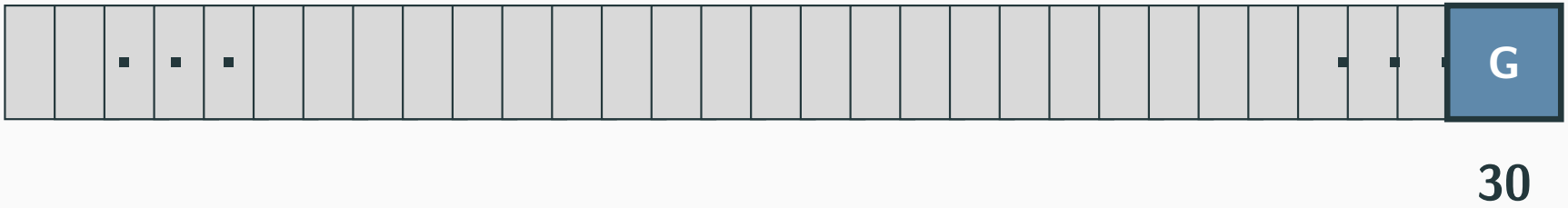
The Process: A Systematic Positional Sweep

Run 20: Gold DB at Position 20



The Process: A Systematic Positional Sweep

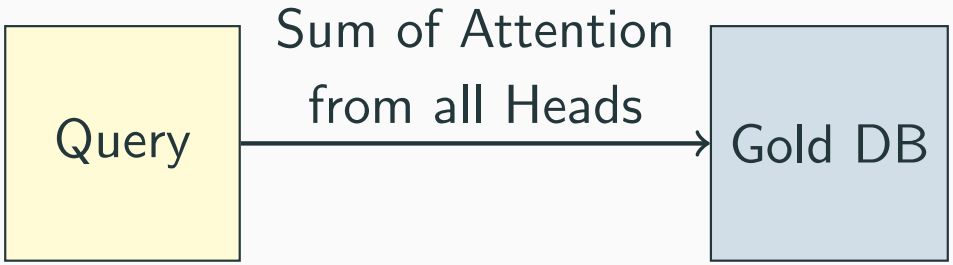
Run 30: Gold DB at Position 30



First Look: Where Does the Model's Total Attention Go?

The Measurement: Tracking Total Attention

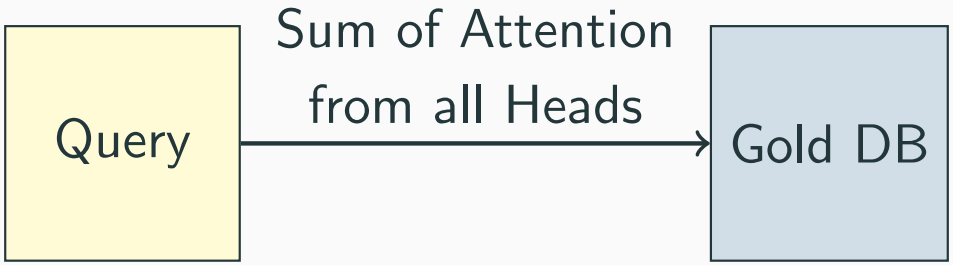
For our first analysis, we ignore individual heads. We simply sum the `total attention` from the query to the Gold DB across *all 1024 heads* for each of the 30 positions.



First Look: Where Does the Model's Total Attention Go?

The Measurement: Tracking Total Attention

For our first analysis, we ignore individual heads. We simply sum the `total attention` from the query to the Gold DB across *all 1024 heads* for each of the 30 positions.



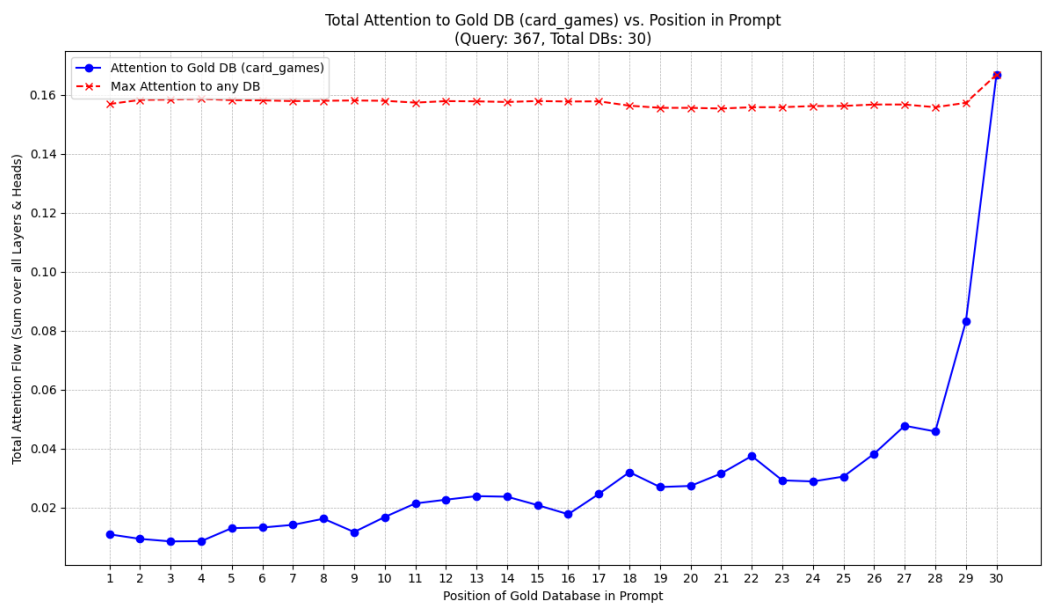
The Question

How does this total attention score change as we move the Gold DB's position from 1 to 30?

Finding: A Pervasive "J-Curve" Positional Bias

Key Finding: A Strong Recency Bias

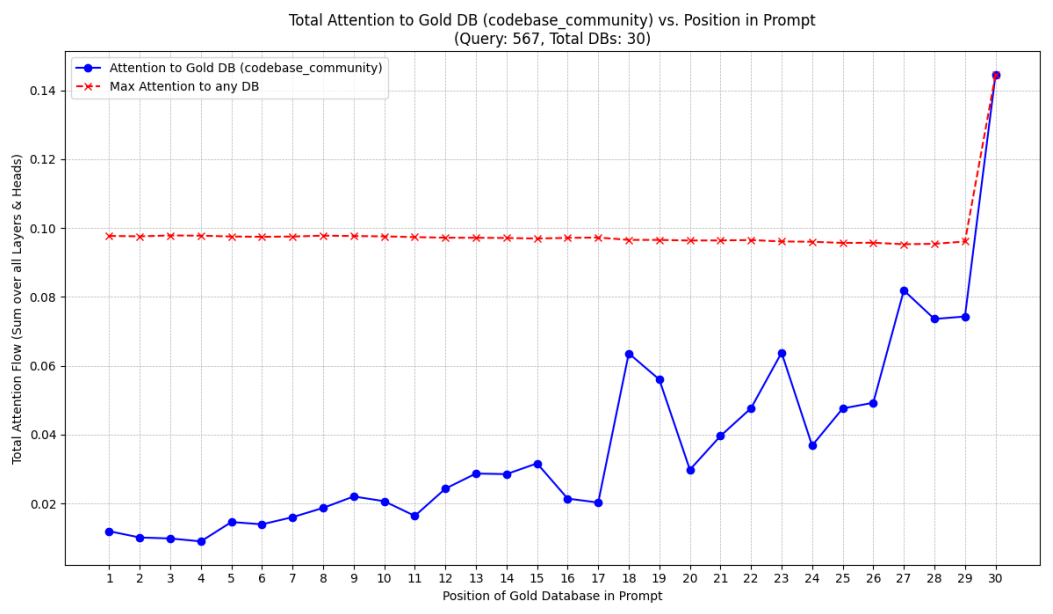
This pattern demonstrates a powerful architectural bias. The model consistently pays the most attention to the final items in its context, largely ignoring information presented in the middle. This raises a critical question: is the model truly reasoning, or just using a simplistic heuristic?



Finding: A Pervasive "J-Curve" Positional Bias

Key Finding: A Strong Recency Bias

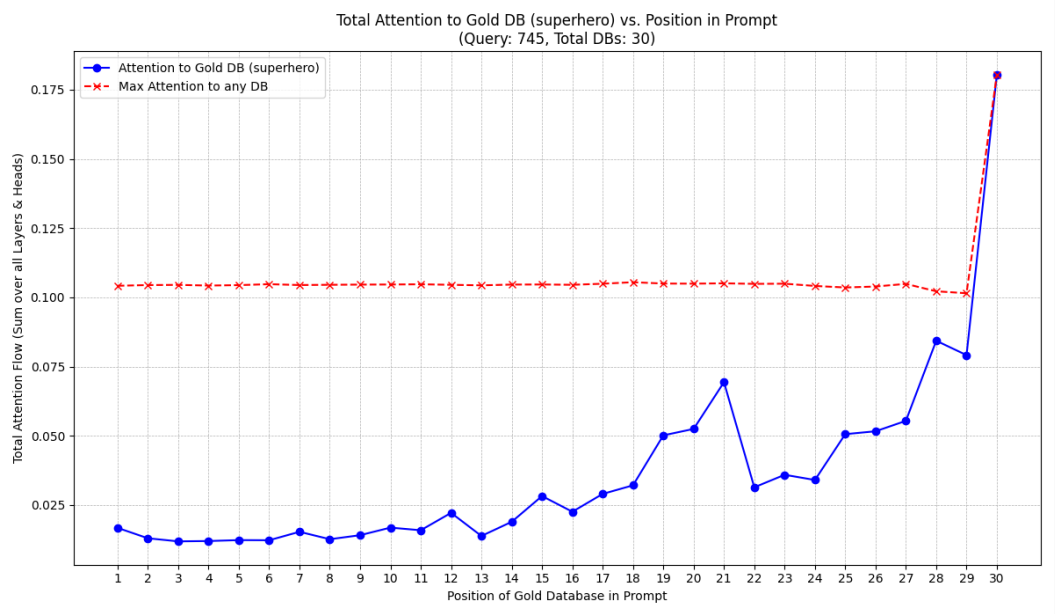
This pattern demonstrates a powerful architectural bias. The model consistently pays the most attention to the final items in its context, largely ignoring information presented in the middle. This raises a critical question: is the model truly reasoning, or just using a simplistic heuristic?



Finding: A Pervasive "J-Curve" Positional Bias

Key Finding: A Strong Recency Bias

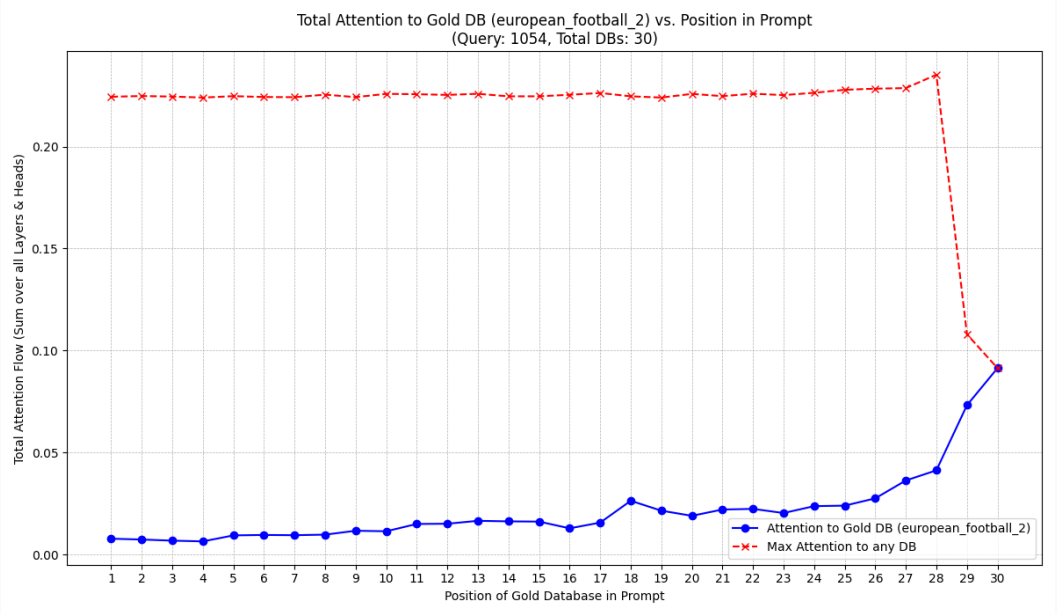
This pattern demonstrates a powerful architectural bias. The model consistently pays the most attention to the final items in its context, largely ignoring information presented in the middle. This raises a critical question: is the model truly reasoning, or just using a simplistic heuristic?



Finding: A Pervasive "J-Curve" Positional Bias

Key Finding: A Strong Recency Bias

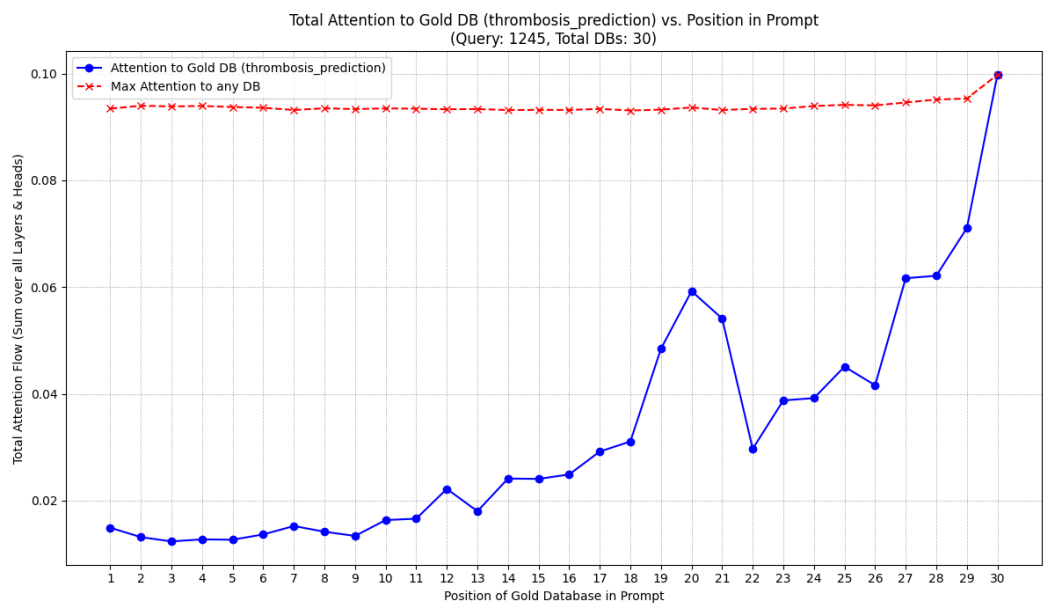
This pattern demonstrates a powerful architectural bias. The model consistently pays the most attention to the final items in its context, largely ignoring information presented in the middle. This raises a critical question: is the model truly reasoning, or just using a simplistic heuristic?



Finding: A Pervasive "J-Curve" Positional Bias

Key Finding: A Strong Recency Bias

This pattern demonstrates a powerful architectural bias. The model consistently pays the most attention to the final items in its context, largely ignoring information presented in the middle. This raises a critical question: is the model truly reasoning, or just using a simplistic heuristic?



Digging Deeper: Identifying the "Reasoning Heads"

The Next Step: Head-Level Analysis

Instead of summing attention across all heads, we analyze each of the 1024 attention heads individually to see which ones are effectively identifying the Gold DB.

The Goal

Our aim is to create a `Success Rate Matrix` (32 layers x 32 heads) that reveals which specific heads are consistently effective at this task, regardless of the Gold DB's position.

Step 1: Analyzing a Single Run (e.g., Run 1)

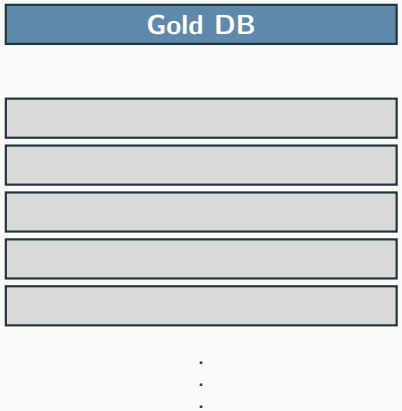
Input
(Gold DB at Pos 1)

Model's Attention Heads
(32 Layers x 32 Heads)

Output
("Win Matrix")

Step 1: Analyzing a Single Run (e.g., Run 1)

Input
(Gold DB at Pos 1)

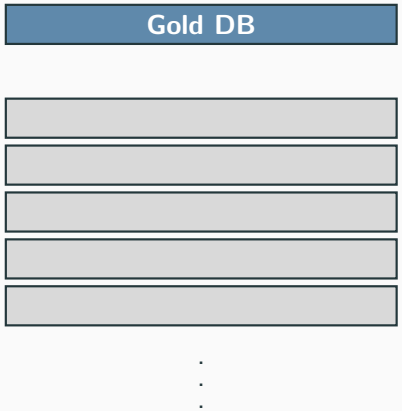


Model's Attention Heads
(32 Layers x 32 Heads)

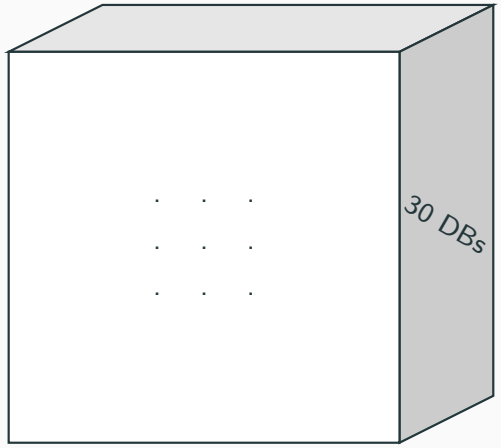
Output
("Win Matrix")

Step 1: Analyzing a Single Run (e.g., Run 1)

Input
(Gold DB at Pos 1)



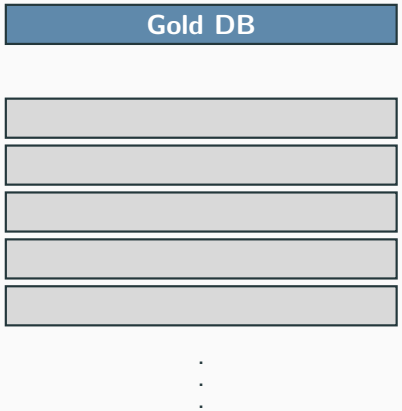
Model's Attention Heads
(32 Layers x 32 Heads)



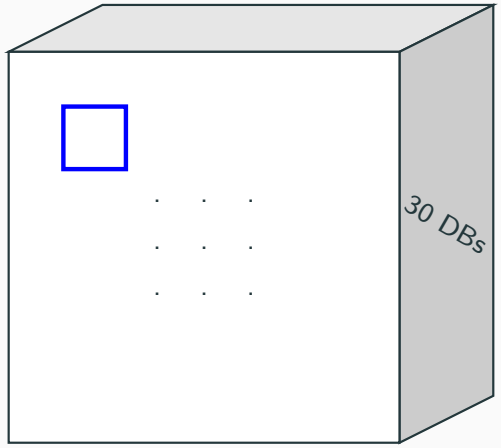
Output
("Win Matrix")

Step 1: Analyzing a Single Run (e.g., Run 1)

Input
(Gold DB at Pos 1)

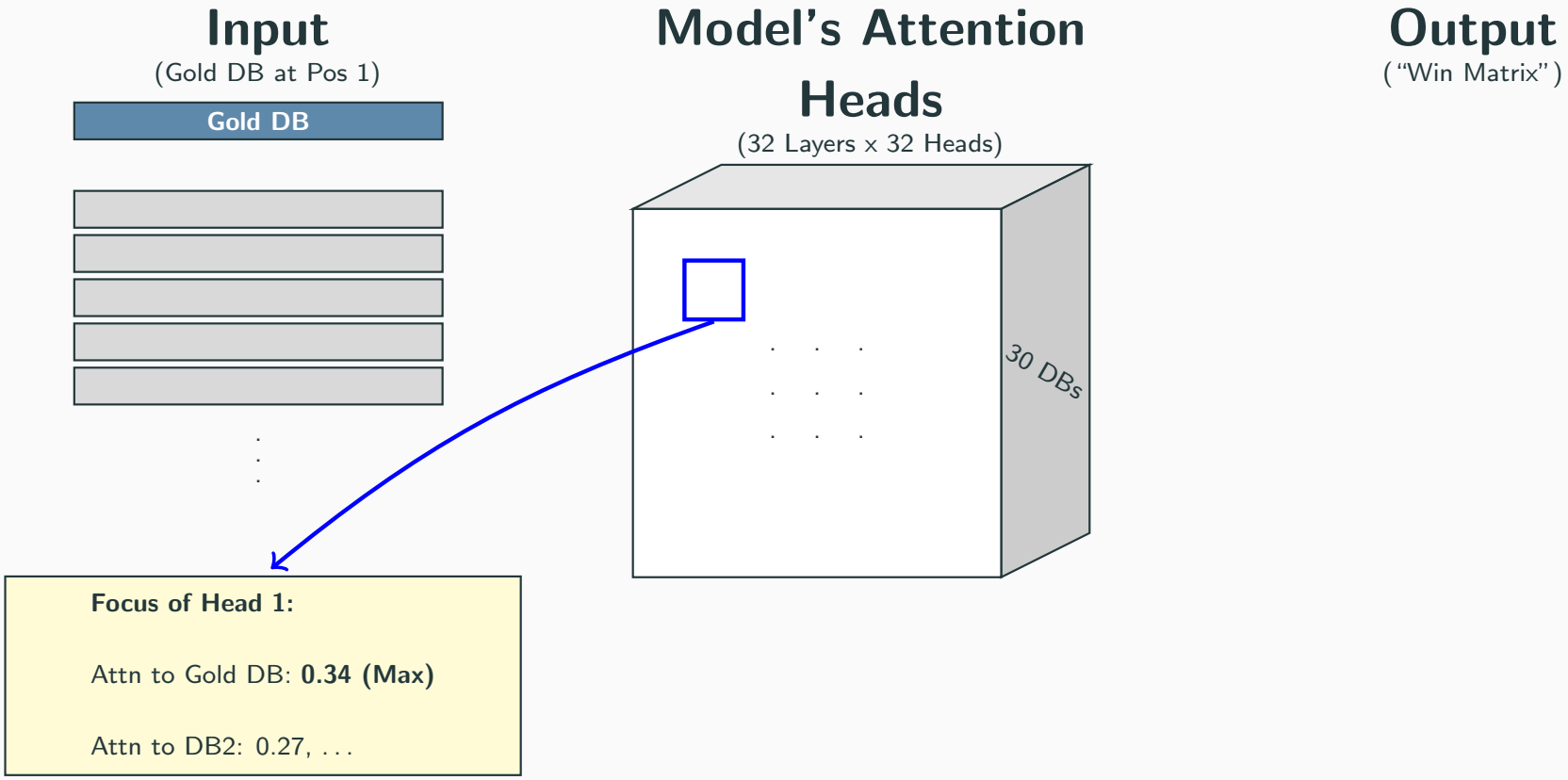


Model's Attention Heads
(32 Layers x 32 Heads)

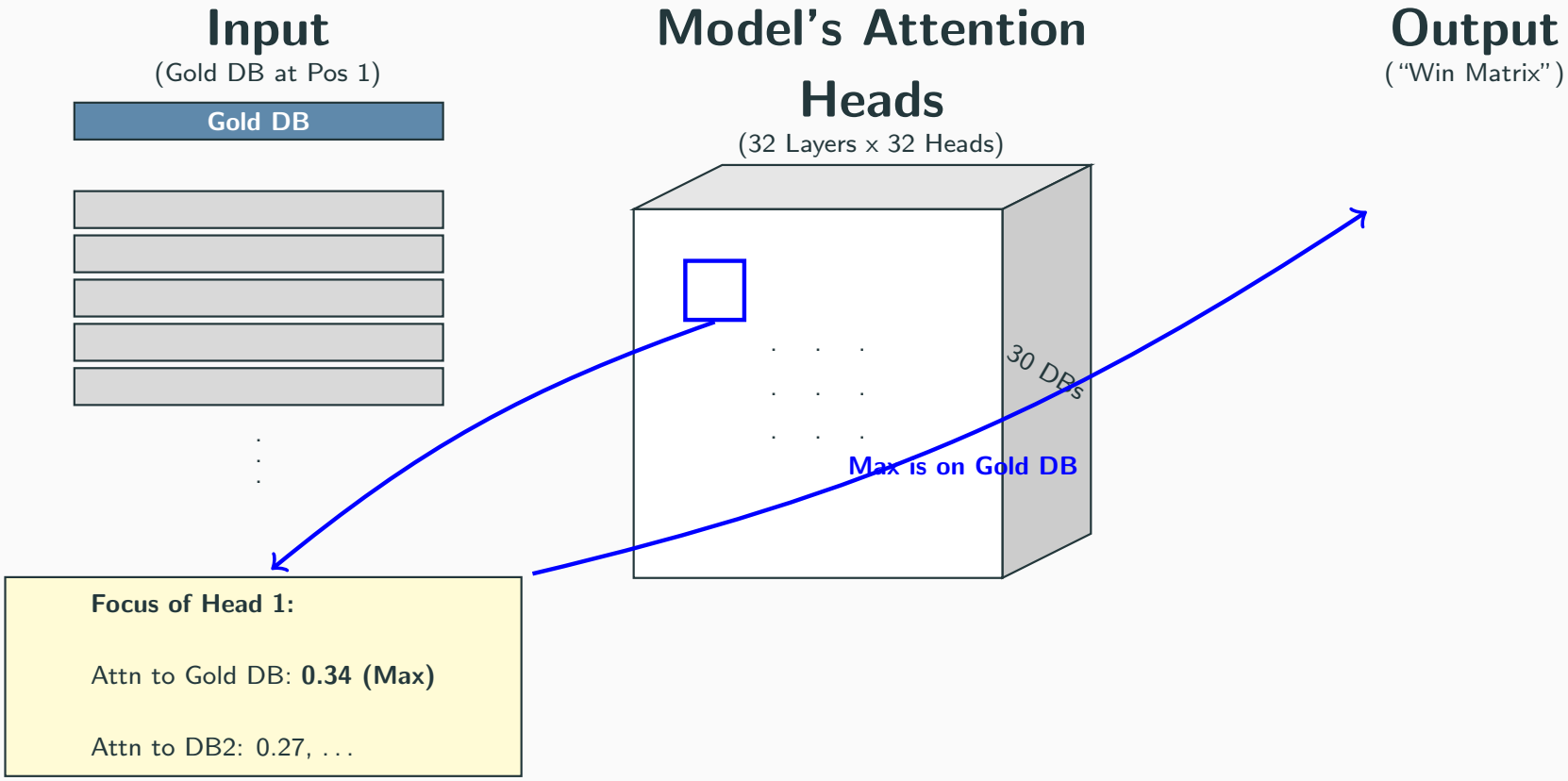


Output
("Win Matrix")

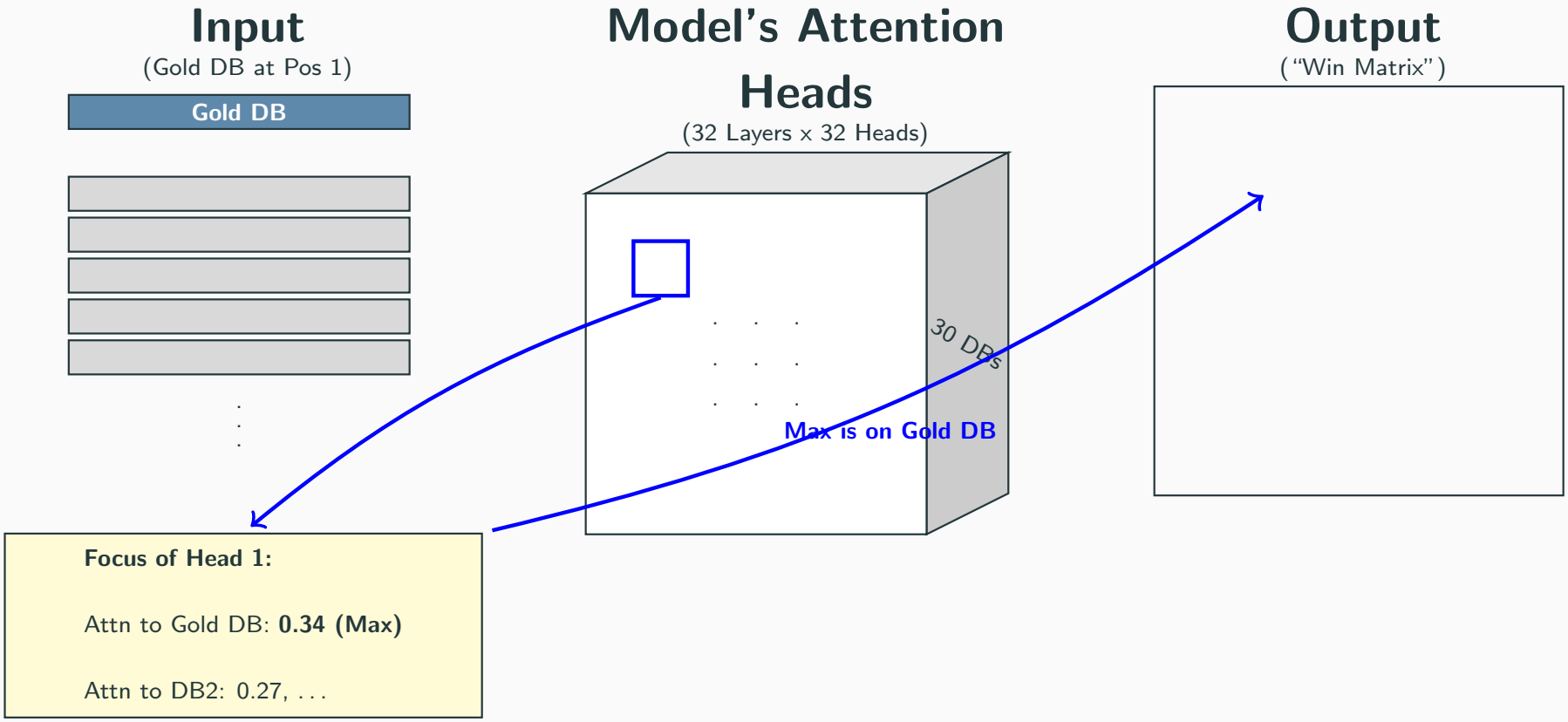
Step 1: Analyzing a Single Run (e.g., Run 1)



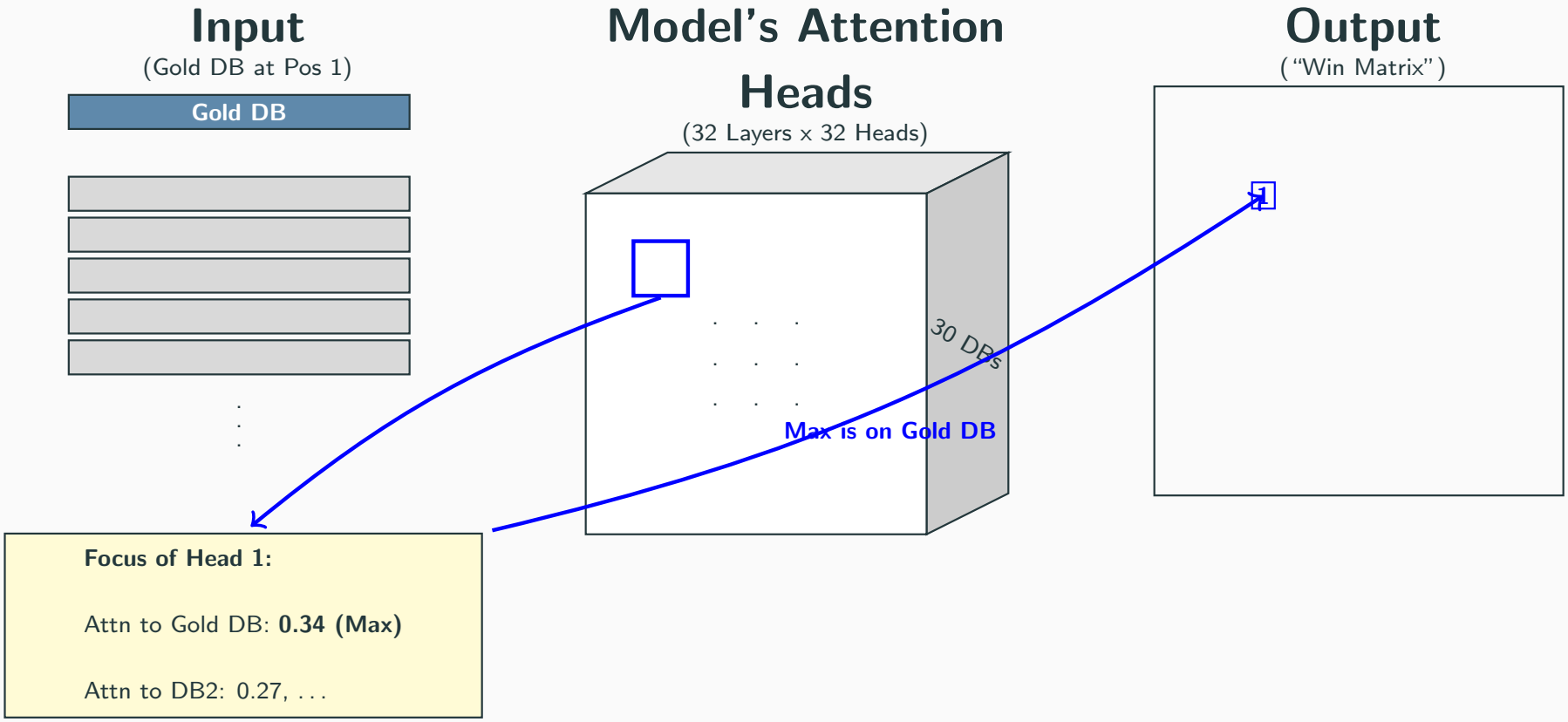
Step 1: Analyzing a Single Run (e.g., Run 1)



Step 1: Analyzing a Single Run (e.g., Run 1)



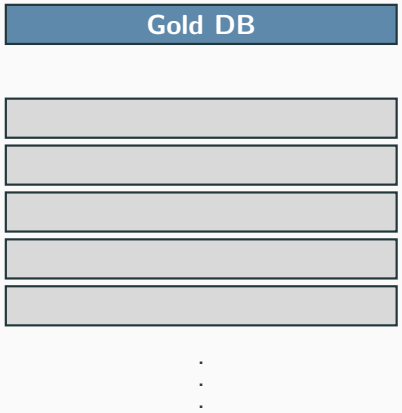
Step 1: Analyzing a Single Run (e.g., Run 1)



Step 1: Analyzing a Single Run (e.g., Run 1)

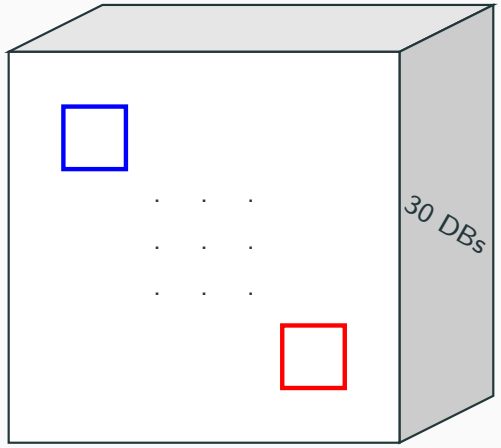
Input

(Gold DB at Pos 1)



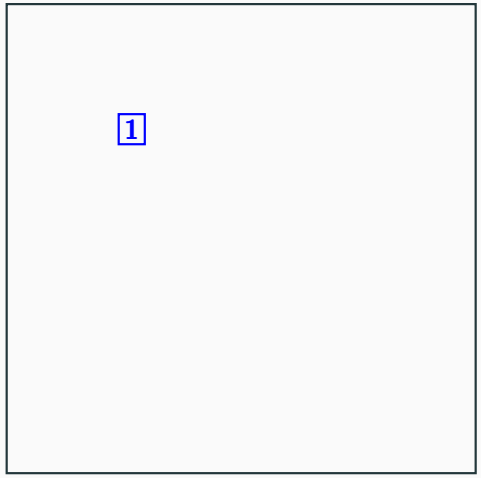
Model's Attention Heads

(32 Layers x 32 Heads)



Output

("Win Matrix")

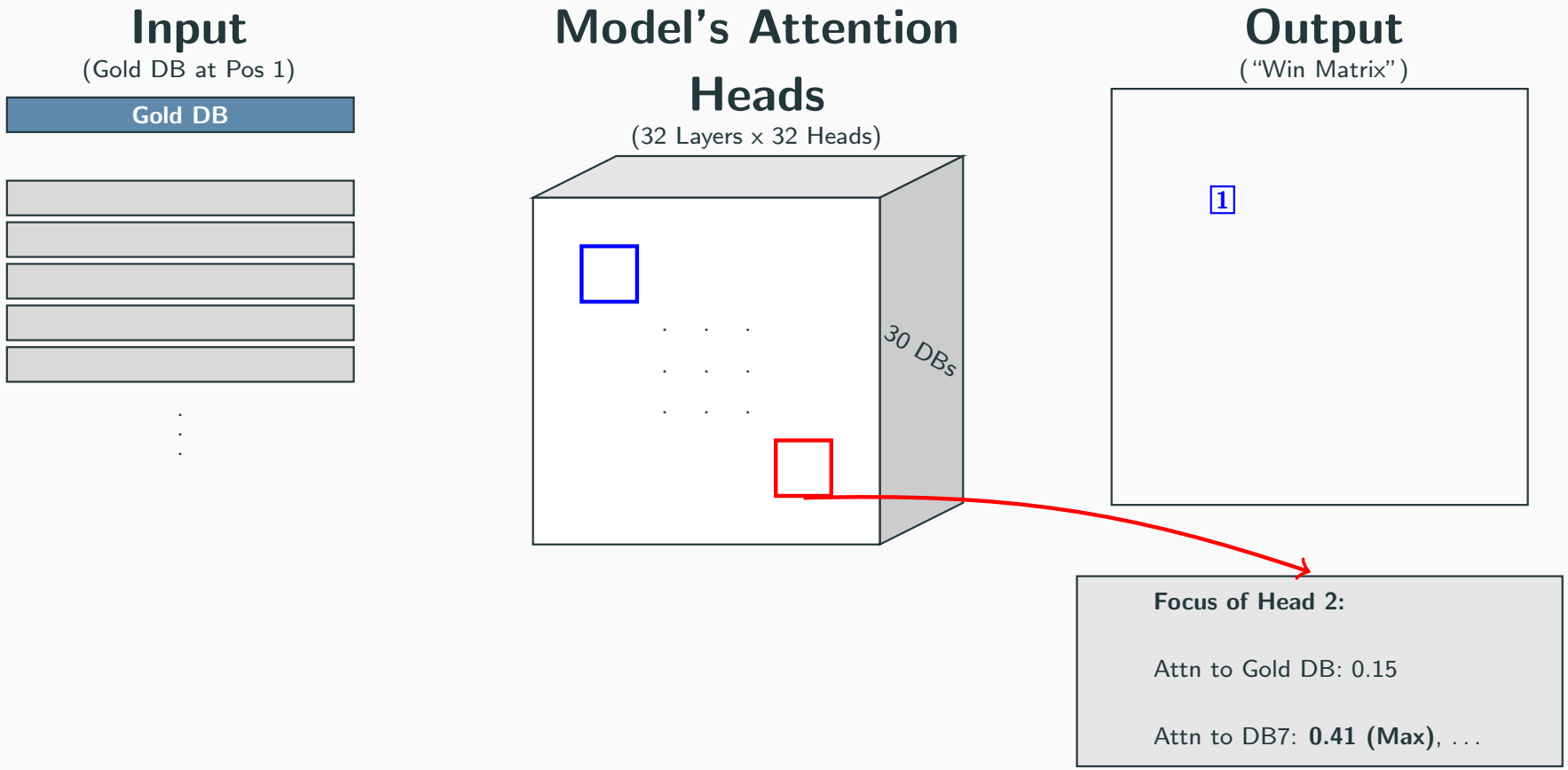


Focus of Head 2:

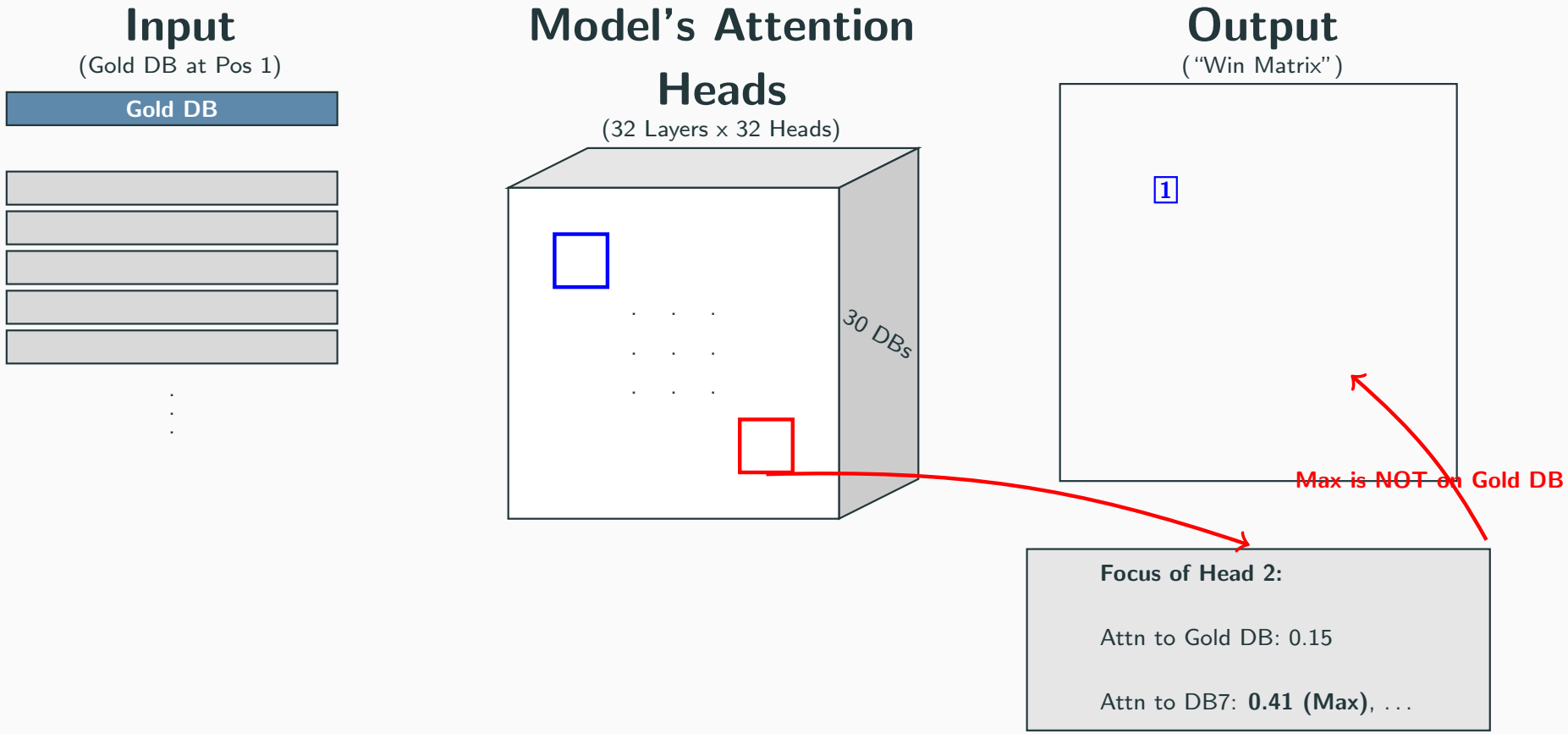
Attn to Gold DB: 0.15

Attn to DB7: **0.41 (Max)**, ...

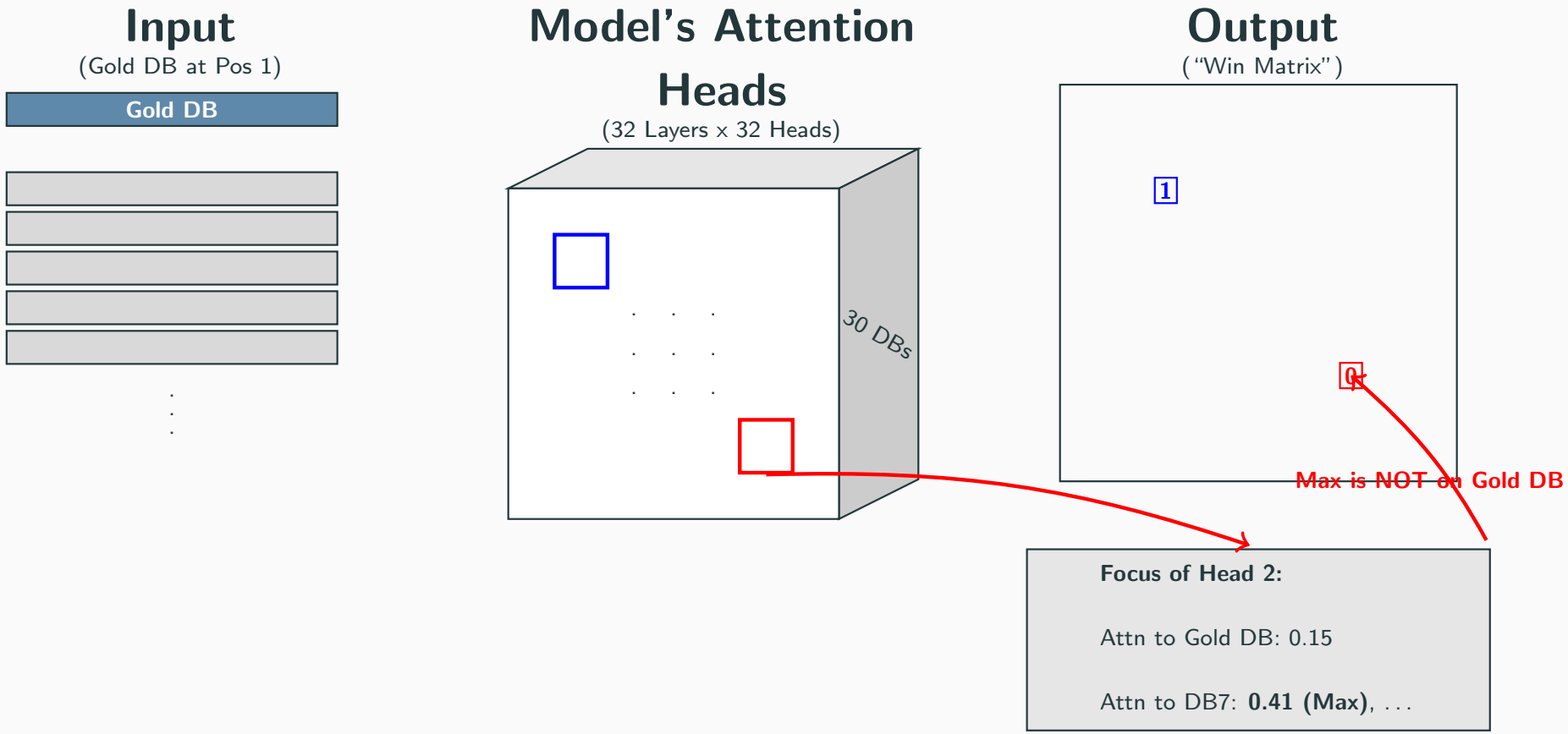
Step 1: Analyzing a Single Run (e.g., Run 1)



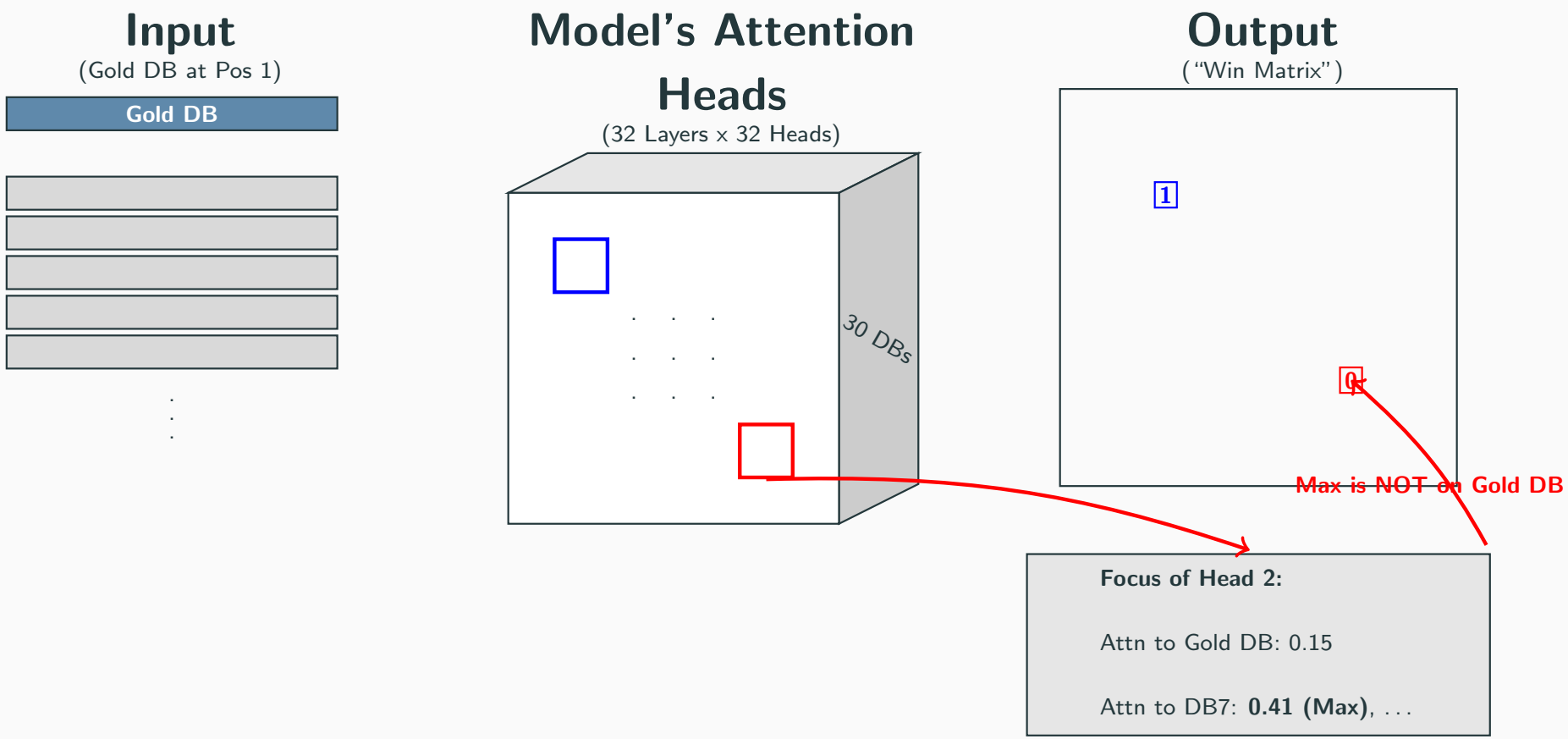
Step 1: Analyzing a Single Run (e.g., Run 1)



Step 1: Analyzing a Single Run (e.g., Run 1)



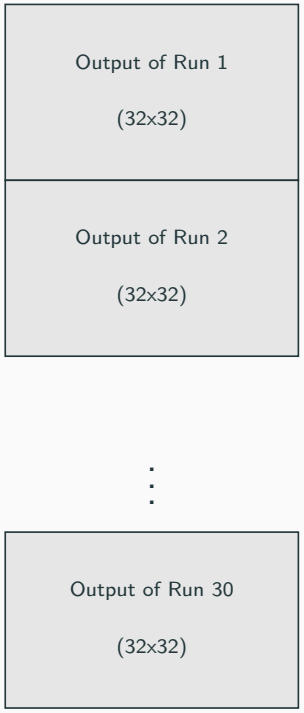
Step 1: Analyzing a Single Run (e.g., Run 1)



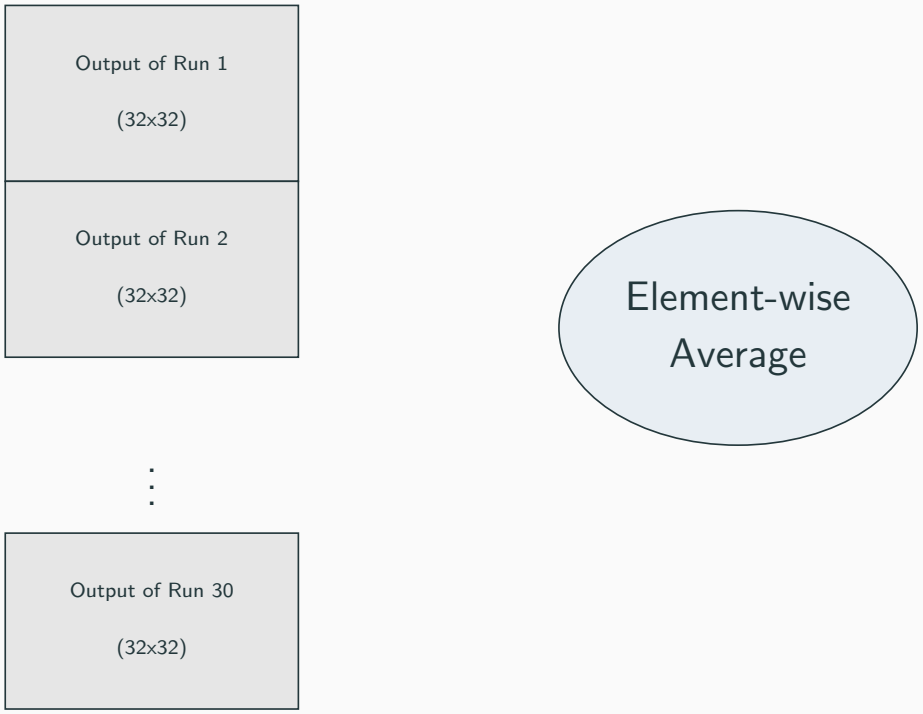
The Process

We repeat this check for all 1024 heads, filling the matrix with '1's and '0's. This completes the matrix for a single run.

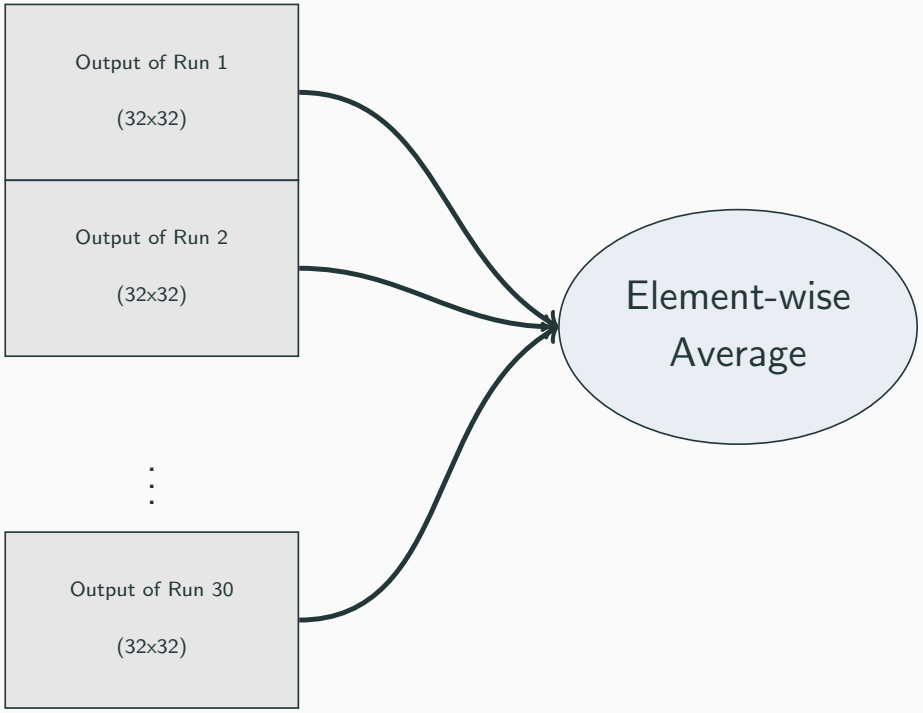
Step 2: Averaging 30 Runs to Find the Signal



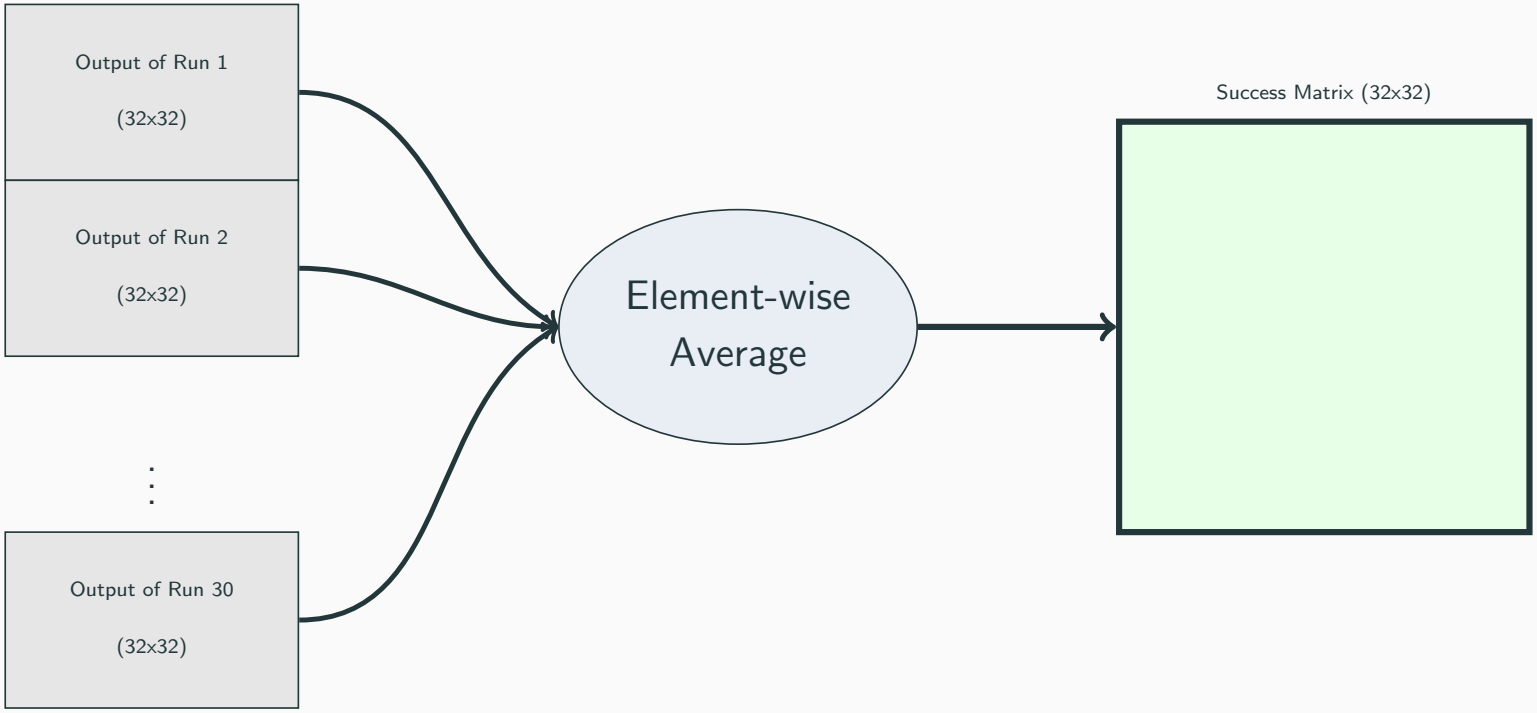
Step 2: Averaging 30 Runs to Find the Signal



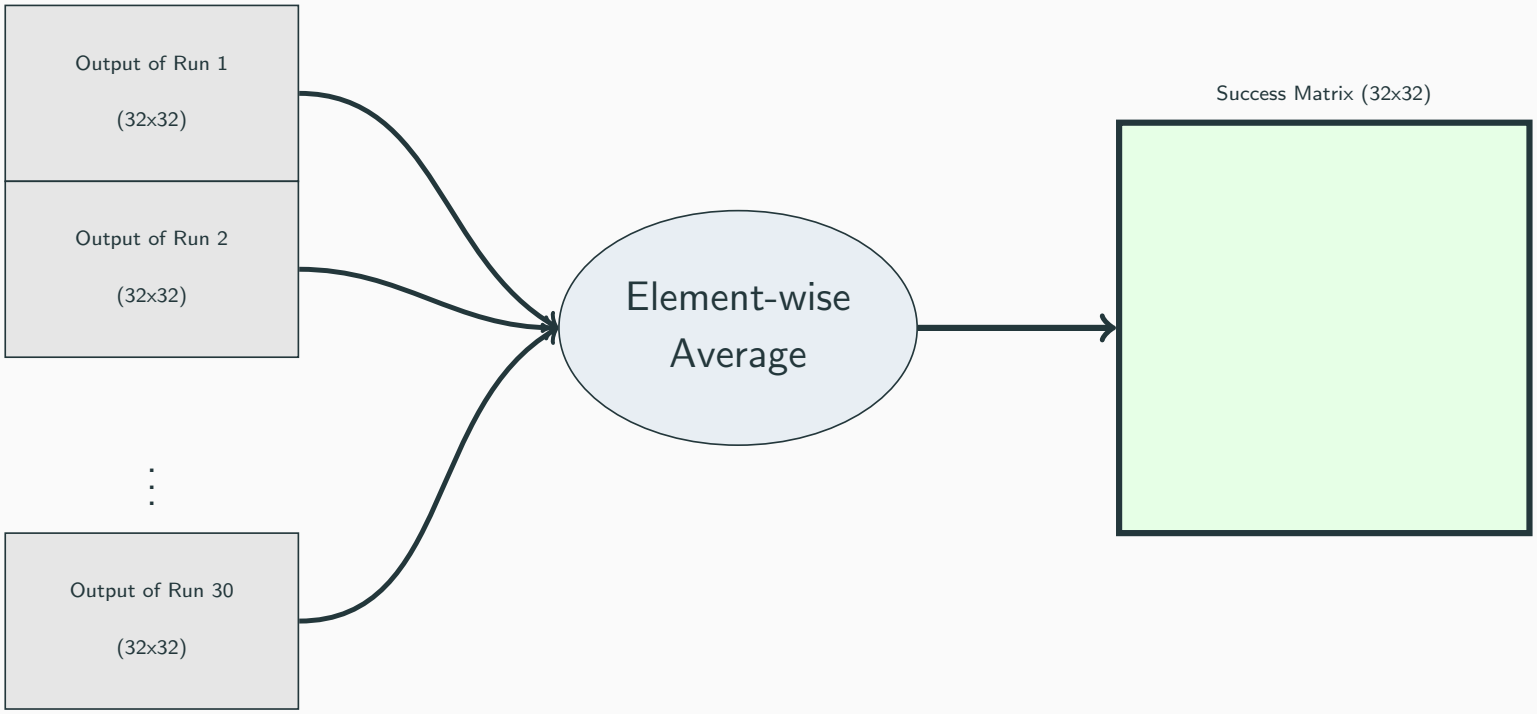
Step 2: Averaging 30 Runs to Find the Signal



Step 2: Averaging 30 Runs to Find the Signal



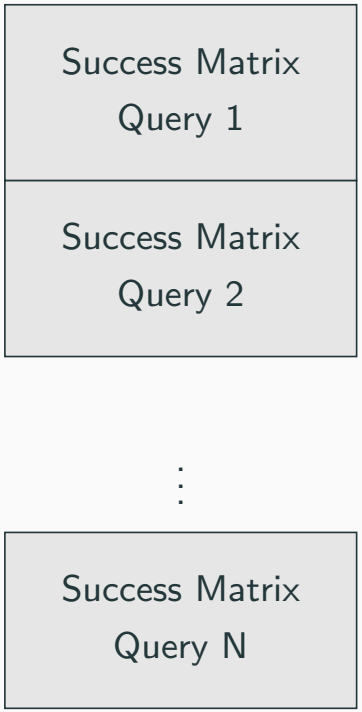
Step 2: Averaging 30 Runs to Find the Signal



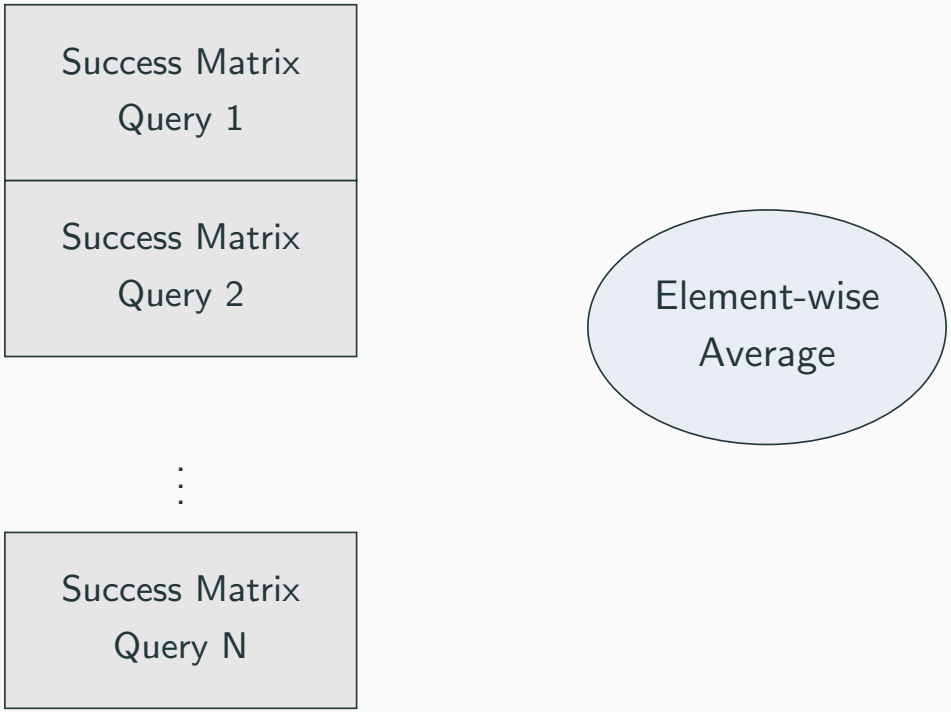
The Transformation

This process transforms 30 noisy, binary matrices into a single `Success Rate Matrix`. Each cell now holds a score from 0.0 to 1.0, representing that head's reliability for this specific query.

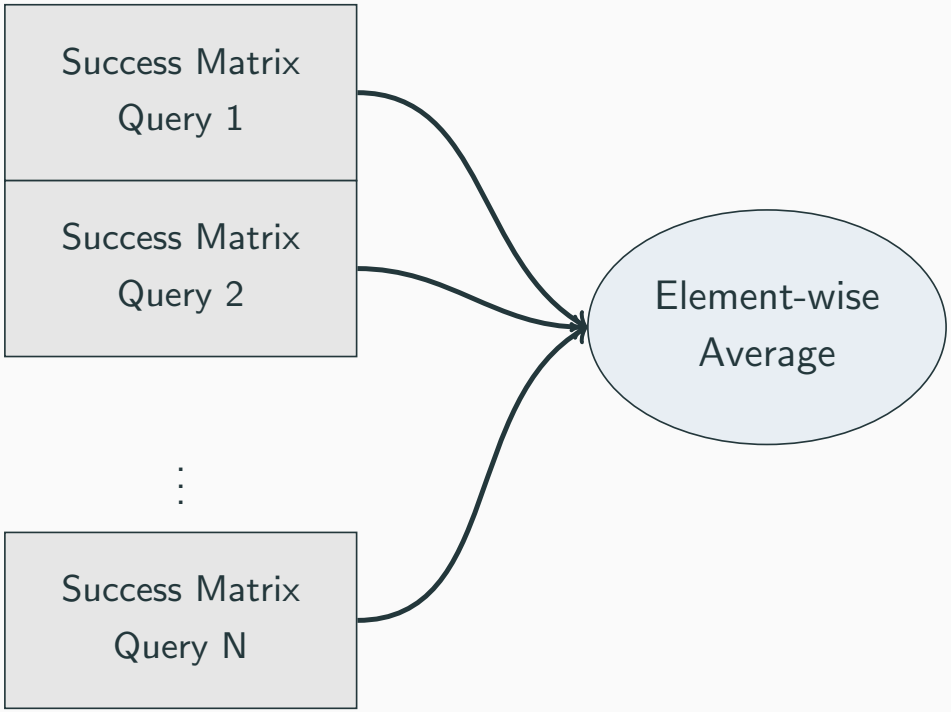
Step 3: Generalizing Across Multiple Queries



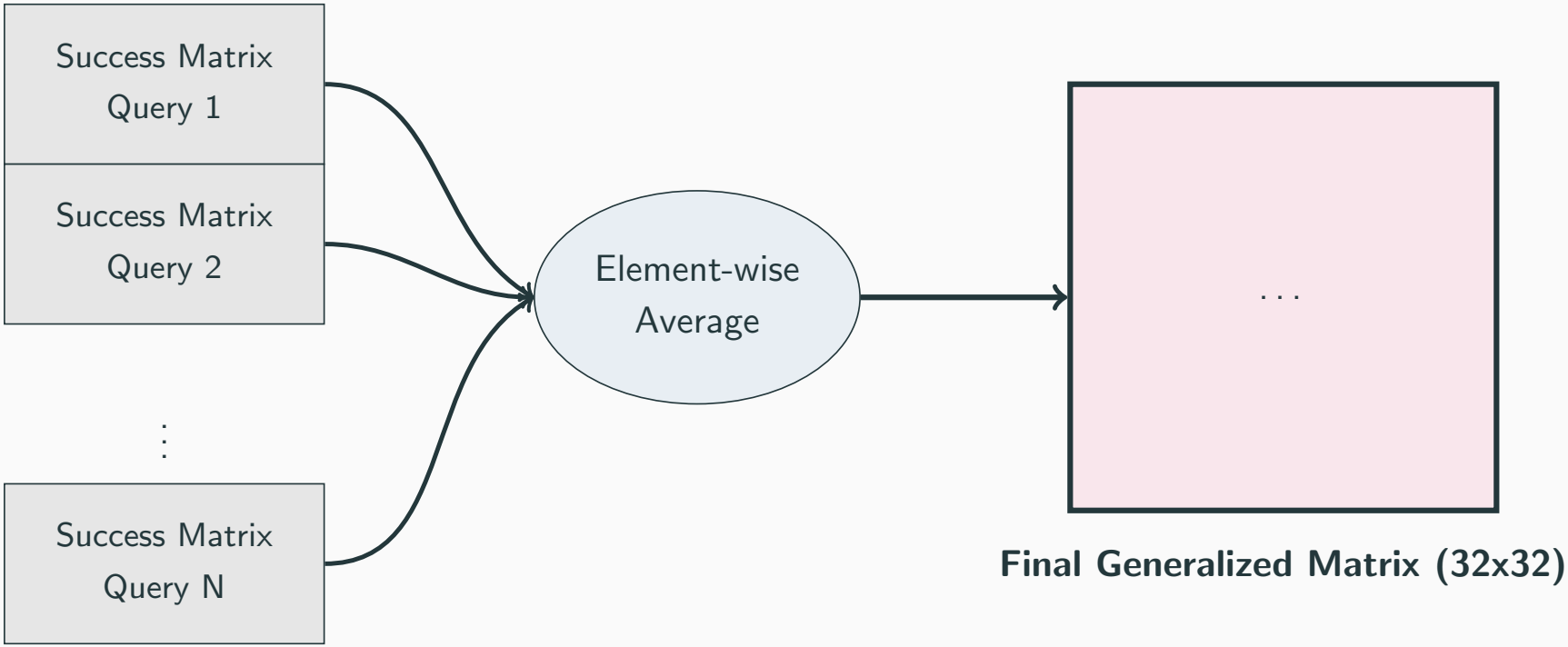
Step 3: Generalizing Across Multiple Queries



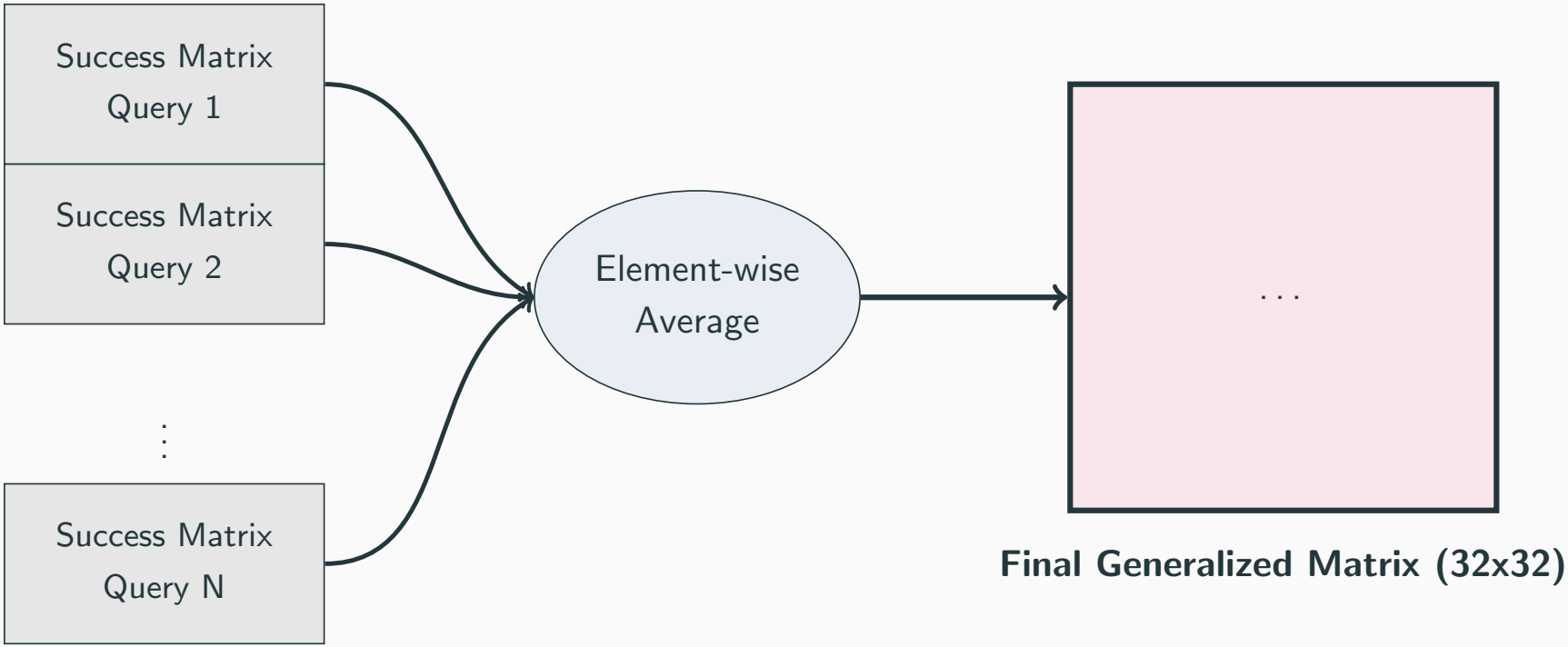
Step 3: Generalizing Across Multiple Queries



Step 3: Generalizing Across Multiple Queries



Step 3: Generalizing Across Multiple Queries



The Final Result: A Map of the "Reasoning Engine"

This final heatmap reveals the small, stable set of attention heads that are consistently effective at this task, regardless of the specific query. These are the core components of the model's reasoning ability.

Findings: The Stable "Reasoning Engine" Emerges

Aggregated Result for N = 10 Queries

With 10 queries, a noisy pattern starts to emerge. Certain heads in the middle-to-late layers show higher-than-average success.

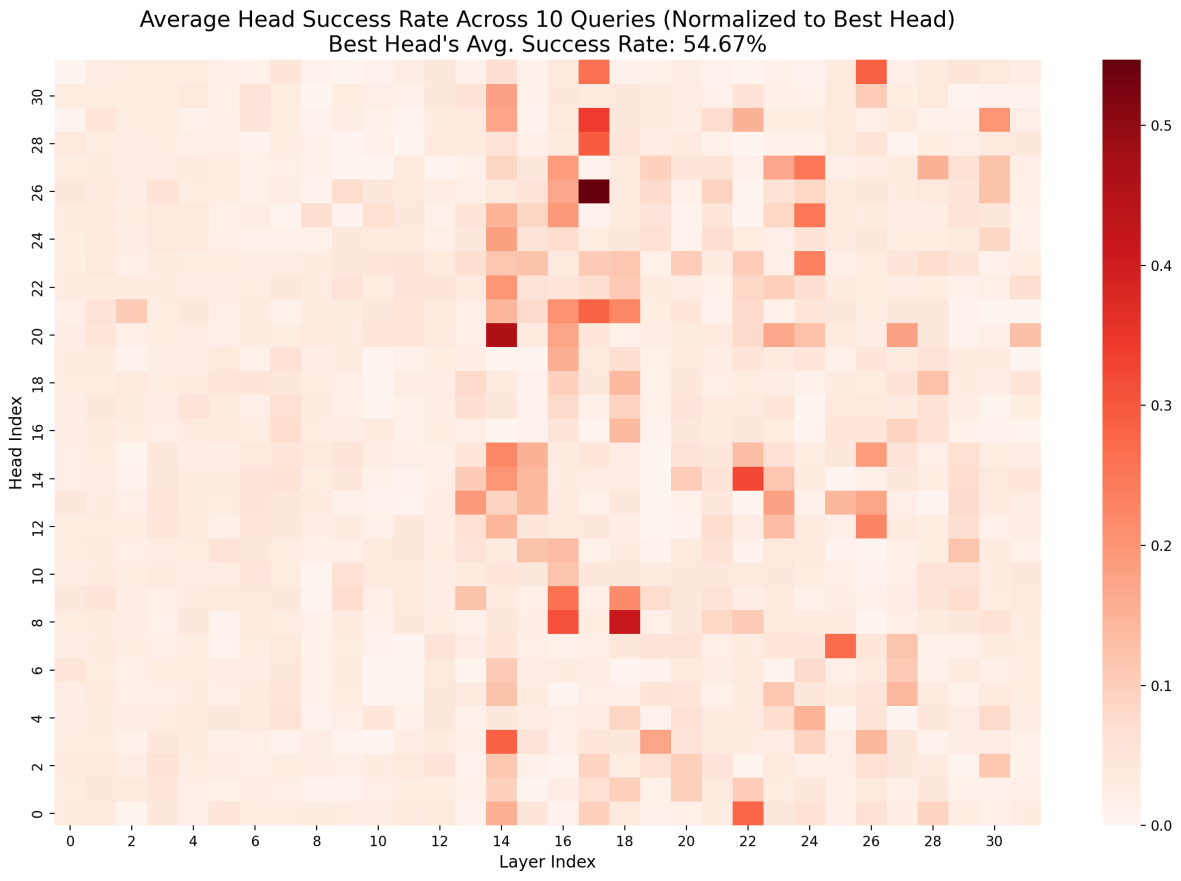


Figure 1: Averaged across 10 distinct queries.

Findings: The Stable "Reasoning Engine" Emerges

Aggregated Result for N = 15 Queries

At 15 queries, the signal gets stronger. The core "hotspots" become more prominent while random noise begins to fade.

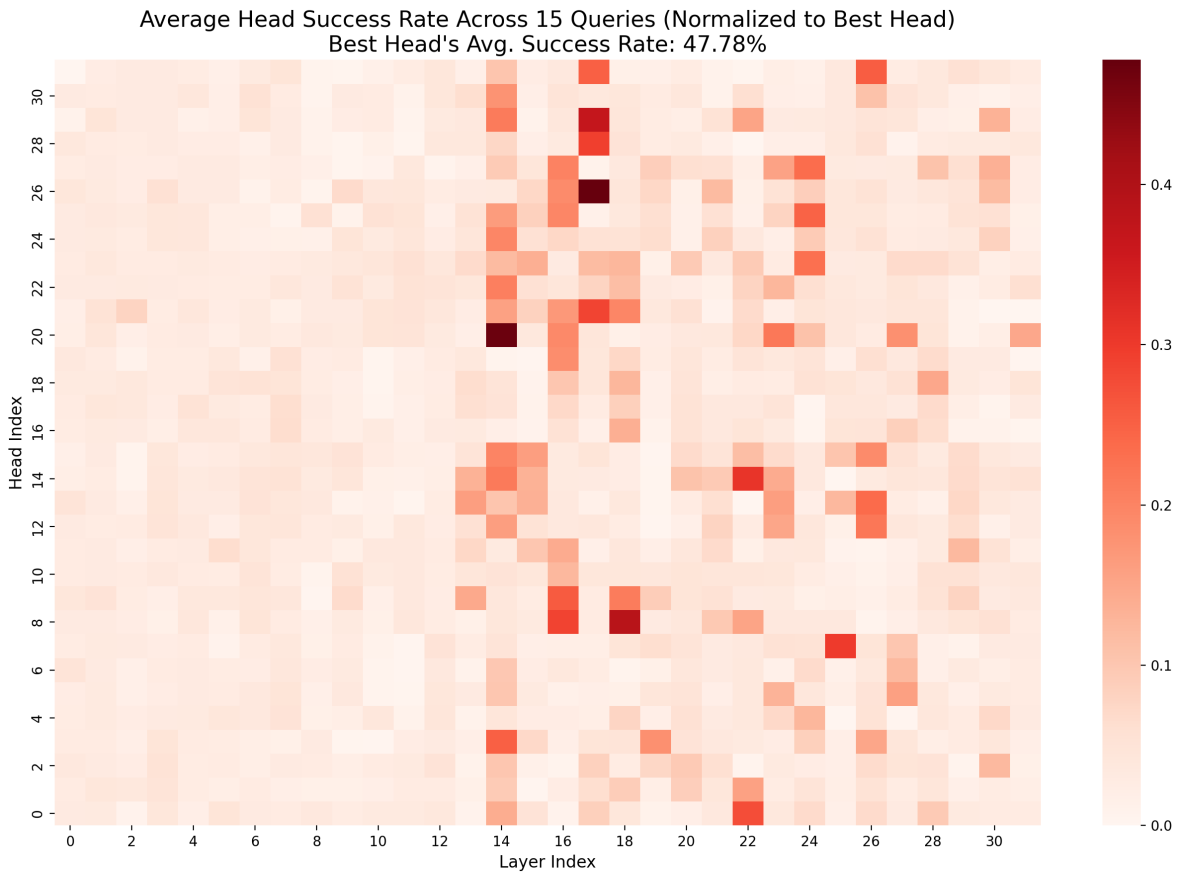


Figure 1: Averaged across 15 distinct queries.

Findings: The Stable "Reasoning Engine" Emerges

Aggregated Result for N = 20 Queries

At 20 queries, the pattern has stabilized. We now have a clear and reliable map of the most important heads for this task.

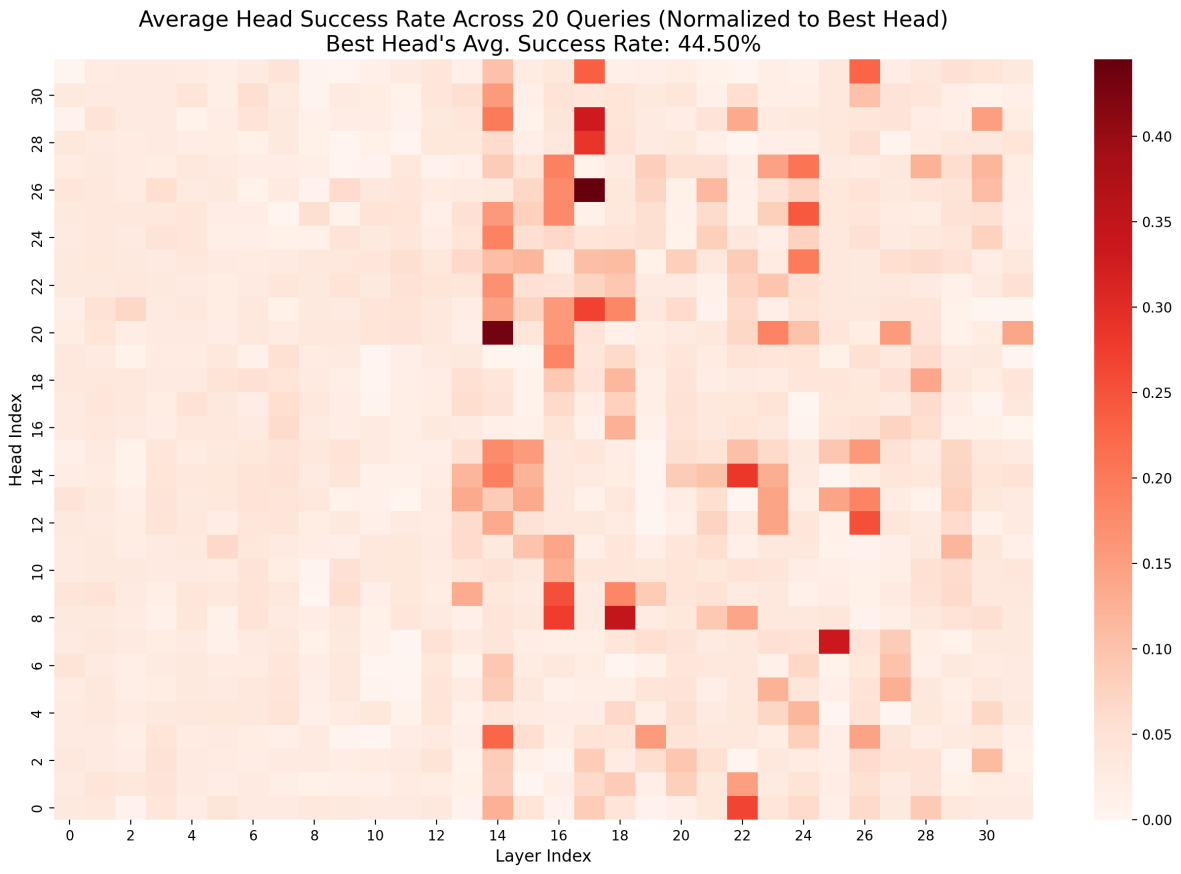
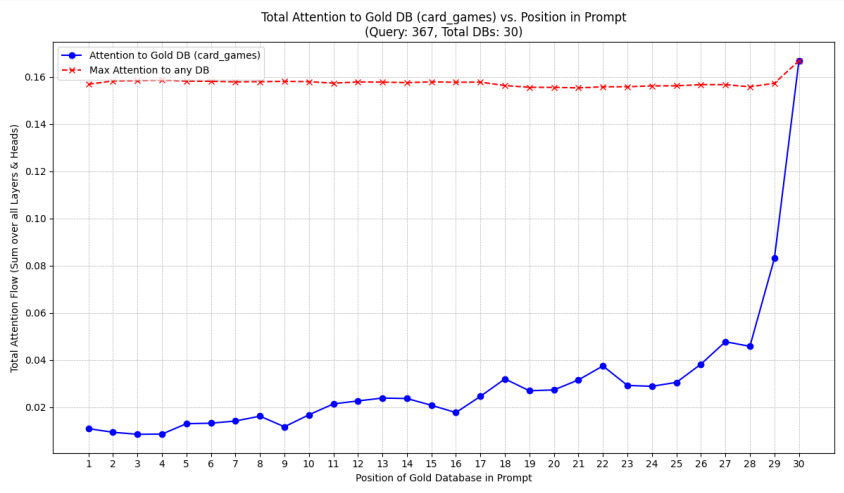


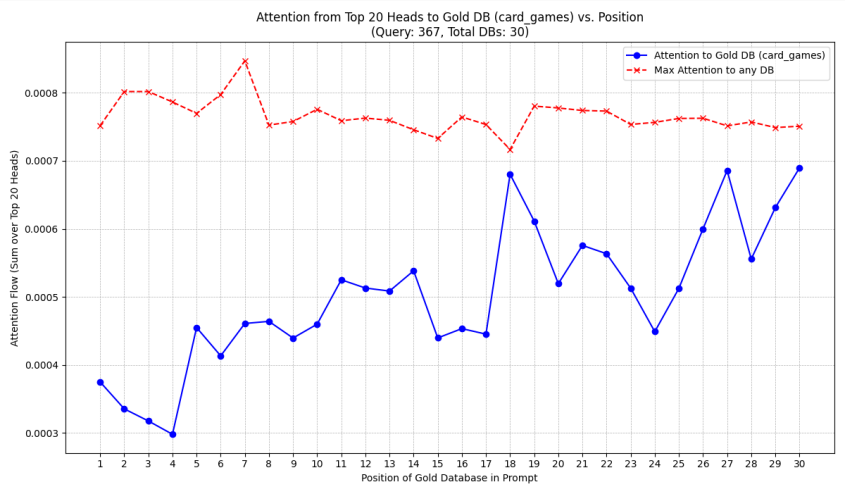
Figure 1: Averaged across 20 distinct queries.

The Revelation: The Suppressed Reasoning Engine in Action

All Heads (Biased)

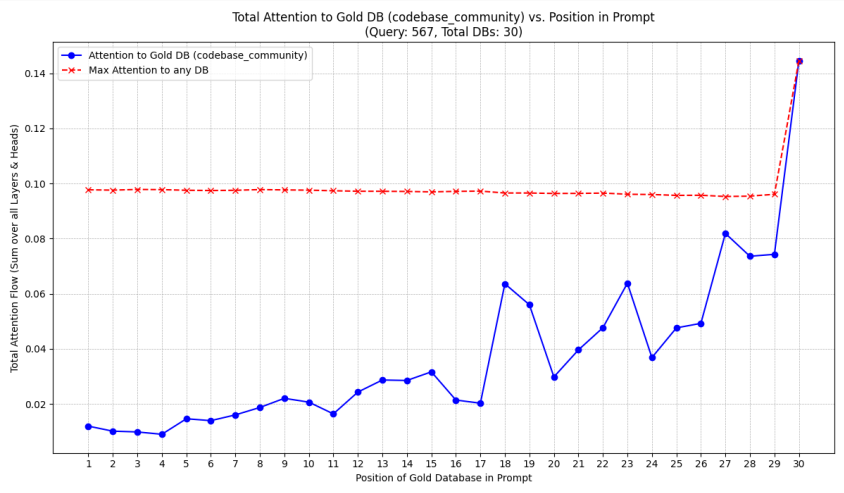


Top 20 Heads (Unbiased)

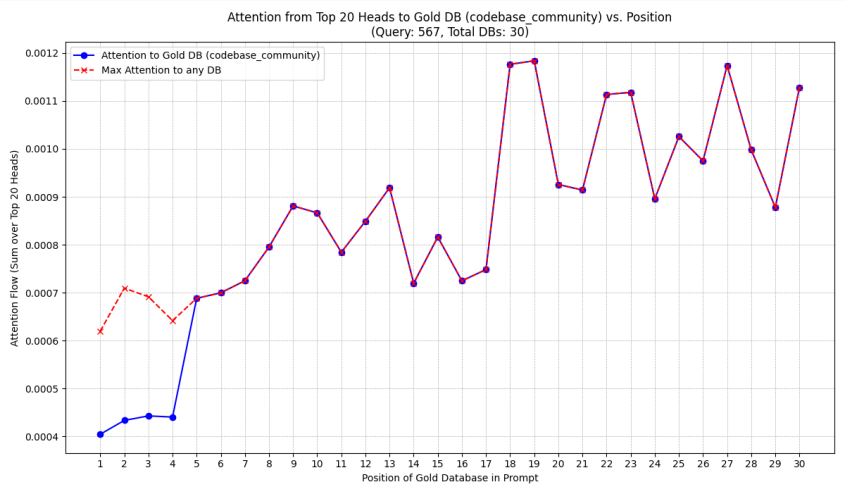


The Revelation: The Suppressed Reasoning Engine in Action

All Heads (Biased)

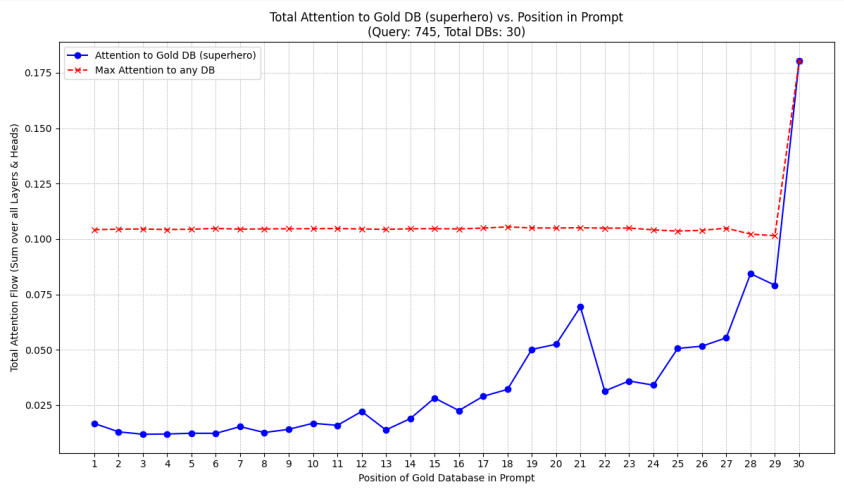


Top 20 Heads (Unbiased)

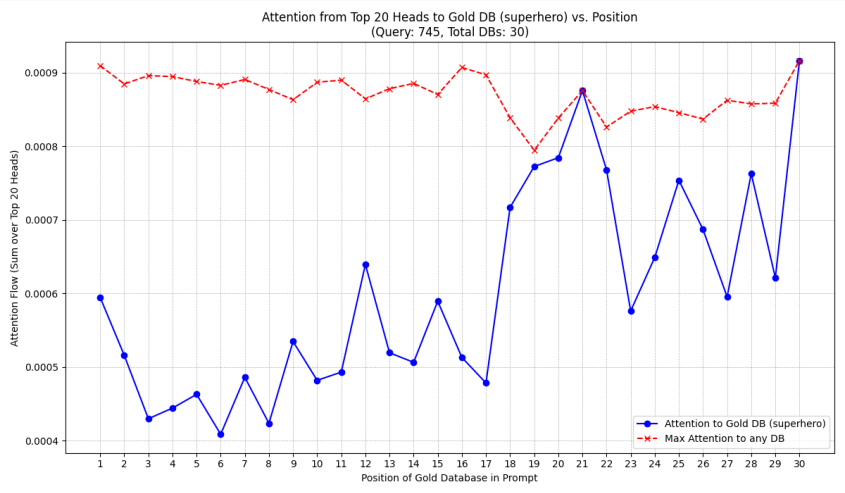


The Revelation: The Suppressed Reasoning Engine in Action

All Heads (Biased)

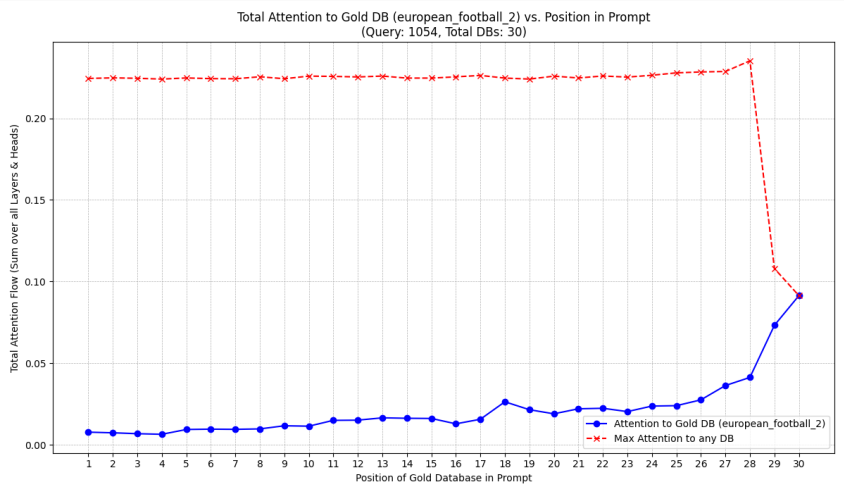


Top 20 Heads (Unbiased)

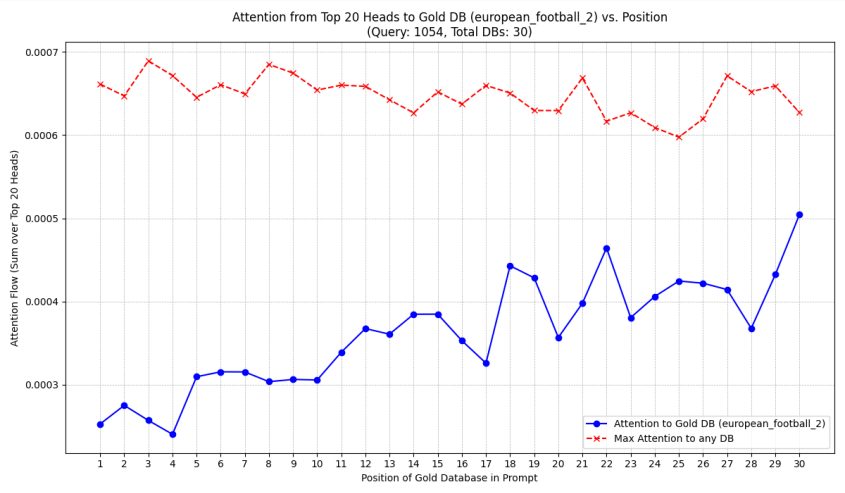


The Revelation: The Suppressed Reasoning Engine in Action

All Heads (Biased)

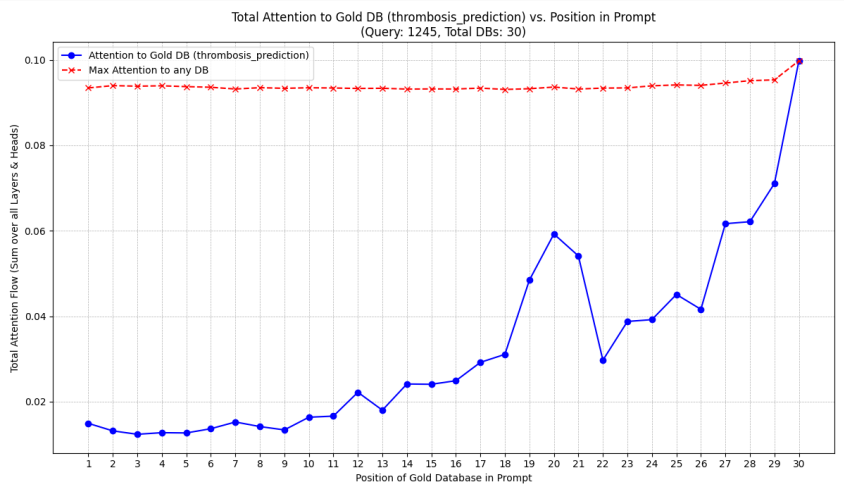


Top 20 Heads (Unbiased)

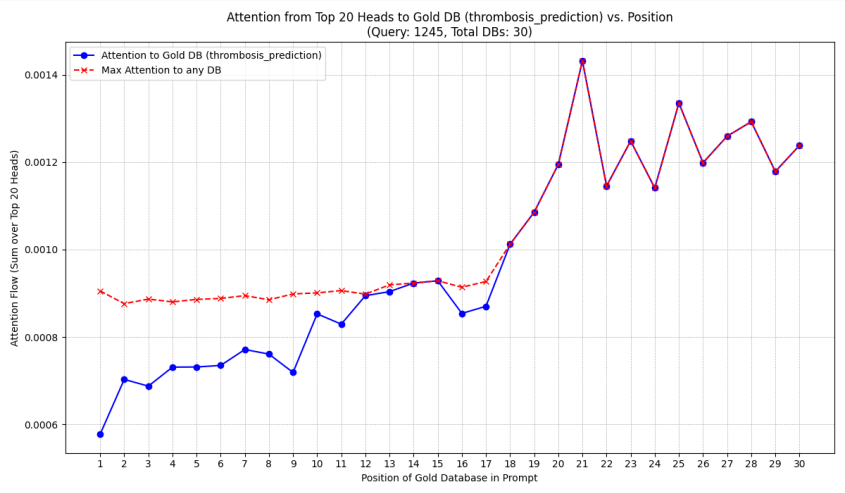


The Revelation: The Suppressed Reasoning Engine in Action

All Heads (Biased)



Top 20 Heads (Unbiased)



Consistent Finding

The pattern is undeniable. In every case, the view from "All Heads" is dominated by a simplistic J-Curve, while the view from the "Top 20 Heads" reveals a sophisticated and better reasoning process.

Part 2: Benchmarking Practical Routing Strategies

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.
    - **Infer:** Perform one inference call to get a 1/0 score

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.
    - **Infer:** Perform one inference call to get a 1/0 score
    - **Store Score:** Add the (DB ID, Score) pair to the list for Query Q.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.
    - **Infer:** Perform one inference call to get a 1/0 score
    - **Store Score:** Add the (DB ID, Score) pair to the list for Query Q.
  - **Rank & Store:** After the inner loop, sort the list of scores to get a final ranked list for Query Q and save it.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.
    - **Infer:** Perform one inference call to get a 1/0 score
    - **Store Score:** Add the (DB ID, Score) pair to the list for Query Q.
  - **Rank & Store:** After the inner loop, sort the list of scores to get a final ranked list for Query Q and save it.
- **Evaluate:** After the outer loop finishes, compare all stored ranked lists against the ground truth to calculate the final Recall@K.

Strategy A: One-by-One (The Classifier)

Algorithm: One-by-One Routing

- **Initialize:** Start with a list of N test queries and M candidate databases.
- **Outer Loop** for each Query Q in the list:
  - Initialize an empty list of scores for Query Q.
  - **Inner Loop** for each Database D_j in the M candidates:
    - **Assemble Prompt:** Create a small prompt containing only Query Q and Database D_j.
    - **Infer:** Perform one inference call to get a 1/0 score
    - **Store Score:** Add the (DB ID, Score) pair to the list for Query Q.
  - **Rank & Store:** After the inner loop, sort the list of scores to get a final ranked list for Query Q and save it.
- **Evaluate:** After the outer loop finishes, compare all stored ranked lists against the ground truth to calculate the final Recall@K.

Key Characteristic

This method is thorough but slow, requiring $N \times M$ total inference calls. It avoids long-context issues by design.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:
   2.1 **Assemble Prompt:** Create one large prompt containing Query Q and *all* candidate database schemas.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:
   - 2.1 **Assemble Prompt:** Create one large prompt containing Query Q and *all* candidate database schemas.
   - 2.2 **Infer:** Perform a *single* inference call to the LLM.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:
   - 2.1 **Assemble Prompt:** Create one large prompt containing Query Q and *all* candidate database schemas.
   - 2.2 **Infer:** Perform a *single* inference call to the LLM.
   - 2.3 **Store Result:** Parse the LLM's output to get a ranked list of database IDs and save it for Query Q.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:
   - 2.1 **Assemble Prompt:** Create one large prompt containing Query Q and *all* candidate database schemas.
   - 2.2 **Infer:** Perform a *single* inference call to the LLM.
   - 2.3 **Store Result:** Parse the LLM's output to get a ranked list of database IDs and save it for Query Q.
3. **Evaluate:** After the loop finishes, compare all stored results against the ground truth to calculate the final Recall@K.

Strategy B: All-in-One (The Re-Ranker)

Algorithm: All-in-One Routing

1. **Initialize:** Start with a list of N test queries.
2. **Loop** for each Query Q in the list:
   - 2.1 **Assemble Prompt:** Create one large prompt containing Query Q and *all* candidate database schemas.
   - 2.2 **Infer:** Perform a *single* inference call to the LLM.
   - 2.3 **Store Result:** Parse the LLM's output to get a ranked list of database IDs and save it for Query Q.
3. **Evaluate:** After the loop finishes, compare all stored results against the ground truth to calculate the final Recall@K.

Key Characteristic

This method is fast, requiring only N total inference calls (one for each query). However, it relies on the LLM's ability to handle long contexts effectively.

Benchmark Results: A Clear Trade-Off Emerges

Model & Mode	SPIDER Dataset			BIRD Dataset		
	R@1	R@5	Time (s)	R@1	R@5	Time (s)
Llama-3.1-8B-Instruct						
One-by-one						
No Examples	68.73%	95.18%	3.74	71.53%	94.65%	2.11
+ Pos Examples	76.98%	96.89%	4.02	83.77%	97.97%	1.71
+ Pos/- Neg Ex	77.94%	97.53%	4.56	85.12%	98.10%	1.91
All-in-one						
No Examples	66.53%	78.72%	1.43	70.64%	78.34%	0.98
+ Pos Examples	69.59%	80.19%	1.58	76.72%	84.10%	1.13
+ Pos/- Neg Ex	66.27%	78.26%	1.75	68.94%	82.40%	1.26

Benchmark Results: A Clear Trade-Off Emerges

	SPIDER Dataset			BIRD Dataset		
Model & Mode	**R@1**	**R@5**	**Time (s)**	**R@1**	**R@5**	**Time (s)**
Qwen2.5-7B-Instruct						
One-by-one						
No Examples	81.90%	97.32%	2.46	88.97%	98.10%	1.83
+ Pos Examples	84.58%	99.35%	3.6	89.45%	98.37%	1.49
+ Pos/- Neg Ex	84.68%	98.82%	2.84	89.65%	98.03%	1.65
All-in-one						
No Examples	57.38%	72.91%	1.06	68.94%	78.68%	0.75
+ Pos Examples	50.10%	59.42%	1.19	45.19%	51.62%	0.83
+ Pos/- Neg Ex	38.75%	47.96%	1.35	39.64%	42.82%	0.96

Benchmark Results: A Clear Trade-Off Emerges

One-by-One: Slow but Accurate

This strategy is consistently superior in accuracy across both models. By avoiding long contexts, it is immune to the positional biases we observed.

All-in-One: Fast but Flawed

While much faster, its performance is significantly lower and can even degrade with more examples. This is a direct consequence of the **J-Curve bias** causing the model to "get lost in the middle."

Retriever Benchmarking

	SPIDER Dataset (1034 queries / 166 DBs)				BIRD Dataset (1534 queries / 80 DBs)			
Retriever	**R@1**	**R@5**	**R@10**	**Time (ms)**	**R@1**	**R@5**	**R@10**	**Time (ms)**
BM25	29.21%	48.26%	50.48%	0.23	29.14%	63.43%	82.07%	0.17
BGE-M3	64.99%	90.52%	95.26%	8.16	69.43%	93.42%	96.35%	7.97
UAE-large-v1	72.44%	92.84%	96.32%	8.06	81.10%	93.48%	96.02%	7.88
ColBERT	73.89%	95.45%	97.87%	0.8	76.00%	93.58%	96.48%	0.76
reasonIR	**85.11%**	**97.68%**	**99.61%**	17.73	**87.76%**	**97.09%**	**98.44%**	17.01

Retriever Benchmarking

	SPIDER Dataset (1034 queries / 166 DBs)				BIRD Dataset (1534 queries / 80 DBs)			
Retriever	**R@1**	**R@5**	**R@10**	**Time (ms)**	**R@1**	**R@5**	**R@10**	**Time (ms)**
BM25	29.21%	48.26%	50.48%	0.23	29.14%	63.43%	82.07%	0.17
BGE-M3	64.99%	90.52%	95.26%	8.16	69.43%	93.42%	96.35%	7.97
UAE-large-v1	72.44%	92.84%	96.32%	8.06	81.10%	93.48%	96.02%	7.88
ColBERT	73.89%	95.45%	97.87%	0.8	76.00%	93.58%	96.48%	0.76
reasonIR	**85.11%**	**97.68%**	**99.61%**	17.73	**87.76%**	**97.09%**	**98.44%**	17.01

Conclusion: `reasonIR` is the Clear Winner

Modern dense retrievers vastly outperform traditional methods.

`reasonIR` stands out as the state-of-the-art.

Part 3: Conclusion

Conclusion: Key Findings and Takeaways

1. **LLMs exhibit a powerful "J-Curve" Positional Bias.**

Conclusion: Key Findings and Takeaways

1. **LLMs exhibit a powerful "J-Curve" Positional Bias.**
2. **A sophisticated "Reasoning Engine" is suppressed within specialist heads.**

Conclusion: Key Findings and Takeaways

1. **LLMs exhibit a powerful "J-Curve" Positional Bias.**
2. **A sophisticated "Reasoning Engine" is suppressed within specialist heads.**
3. **The "One-by-One" routing strategy is superior for accuracy.**

Conclusion: Key Findings and Takeaways

1. **LLMs exhibit a powerful "J-Curve" Positional Bias.**
2. **A sophisticated "Reasoning Engine" is suppressed within specialist heads.**
3. **The "One-by-One" routing strategy is superior for accuracy.**
4. `reasonIR` **is the state-of-the-art for initial retrieval.**

Part 4: Future Work

Future Work

1. Mitigating Positional Bias

Now that we have definitively characterized the "J-Curve" bias, a primary goal is to develop techniques to counteract it.

Future Work

2. Amplifying the "Reasoning Engine"

The discovery of a small set of stable "reasoning heads" is our most powerful finding. We'll explore methods to selectively amplify the influence of these heads.

Future Work

3. Cross-Task Generalization

Our analysis was focused on database routing. A crucial next step is to investigate if these findings generalize to other critical "needle-in-a-haystack" tasks. Do the same specialist heads perform well for document retrieval? Does the J-Curve bias impact tool and function calling in the same way?

Thank You!